knn

November 10, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# assignment folder, e.g. 'icv83551/assignments/assignment1/'
FOLDERNAME = 'icv83551/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/MyDrive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/icv83551/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
/content/drive/My Drive/icv83551/assignments/assignment1/icv83551/datasets
/content/drive/My Drive/icv83551/assignments/assignment1
```

1 k-Nearest Neighbor (kNN) exercise

*Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. The kNN classifier consists of two stages:

- During training, the classifier takes the training data and simply remembers it
- During testing, kNN classifies every test image by comparing to all training images and transfering the labels of the k most similar training examples
- The value of k is cross-validated

In this exercise you will implement these steps and understand the basic Image Classification pipeline, cross-validation, and gain proficiency in writing efficient, vectorized code.

[2]:
```
# Run some setup code for this notebook.

import random
import numpy as np
from icv83551.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the␣
 ↪notebook
# rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

#%load_ext autoreload
#%autoreload 2
```

[3]:
```
# Load the raw CIFAR-10 data.
cifar10_dir = 'icv83551/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause␣
 ↪memory issue)
try:
   del X_train, y_train
   del X_test, y_test
   print('Clear previously loaded data.')
except:
   pass

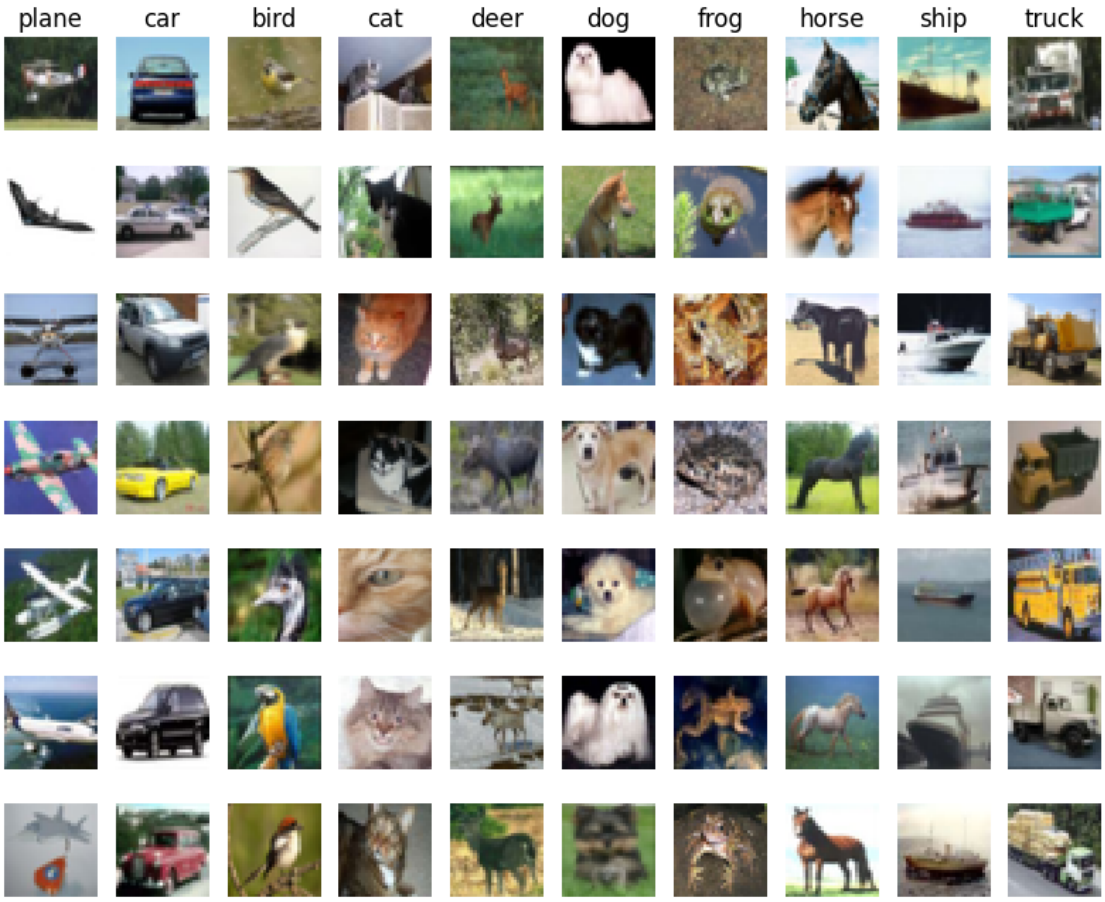
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
```

[4]:
```
# Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']
```

```
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



[5]:
```
# Subsample the data for more efficient code execution in this exercise
num_training = 5000
mask = list(range(num_training))
X_train = X_train[mask]
```

```
y_train = y_train[mask]

num_test = 500
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

# Reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
print(X_train.shape, X_test.shape)
```

```
(5000, 3072) (500, 3072)
```

[6]:
```
import importlib
import icv83551.classifiers.k_nearest_neighbor as knn_module

importlib.reload(knn_module)
KNearestNeighbor = knn_module.KNearestNeighbor

# Create a kNN classifier instance.
# Remember that training a kNN classifier is a noop:
# 'noop' stands for "no operation": in this context, training a kNN classifier
# does not perform any real learning step, it simply stores the labeled data
# so it can be used later to find nearest neighbors during prediction.

# the Classifier simply remembers the data and does no further processing
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
```

We would now like to classify the test data with the kNN classifier. Recall that we can break down this process into two steps:

1. First we must compute the distances between all test examples and all train examples.
2. Given these distances, for each test example we find the k nearest examples and have them vote for the label

Lets begin with computing the distance matrix between all training and test examples. For example, if there are **Ntr** training examples and **Nte** test examples, this stage should result in a **Nte x Ntr** matrix where each element (i,j) is the distance between the i-th test and j-th train example.

Note: For the three distance computations that we require you to implement in this notebook, you may not use the np.linalg.norm() function that numpy provides.

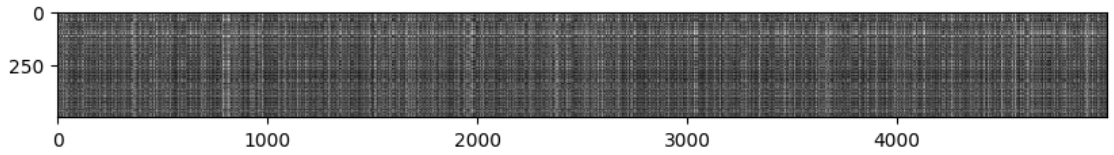
First, open `icv83551/classifiers/k_nearest_neighbor.py` and implement the function `compute_distances_two_loops` that uses a (very inefficient) double loop over all pairs of (test, train) examples and computes the distance matrix one element at a time.

[7]:
```
# Open icv83551/classifiers/k_nearest_neighbor.py and implement
# compute_distances_two_loops.
```

```
# Test your implementation:
dists = classifier.compute_distances_two_loops(X_test)
print(dists.shape)
```

```
(500, 5000)
```

[8]:
```
# We can visualize the distance matrix: each row is a single test example and
# its distances to training examples
plt.imshow(dists, interpolation='none')
plt.show()
```



Inline Question 1

Notice the structured patterns in the distance matrix, where some rows or columns are visibly brighter. (Note that with the default color scheme black indicates low distances while white indicates high distances.)

- What in the data is the cause behind the distinctly bright rows?
- What causes the columns?

Answer:

Bright rows correspond to test images that have relatively large Euclidean distances from most training images. This may be caused by differences in pixel values, such as unusual brightness, colors, or backgrounds.

Similarly, bright columns correspond to training images that are relatively far from most test images.

These patterns reflect differences in the raw pixel values rather than necessarily indicating different semantic classes.

[9]:
```
# Now implement the function predict_labels and run the code below:
# We use k = 1 (which is Nearest Neighbor).
y_test_pred = classifier.predict_labels(dists, k=1)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 137 / 500 correct => accuracy: 0.274000
```

You should expect to see approximately 27% accuracy. Now lets try out a larger `k`, say `k = 5`:

[10]:
```
y_test_pred = classifier.predict_labels(dists, k=5)
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 139 / 500 correct => accuracy: 0.278000
```

You should expect to see a slightly better performance than with `k = 1`.

Inline Question 2

We can also use other distance metrics such as L1 distance. For pixel values $p_{ij}^{(k)}$ at location (i, j) of some image I_k,

the mean μ across all pixels over all images is

$$\mu = \frac{1}{nhw} \sum_{k=1}^{n} \sum_{i=1}^{h} \sum_{j=1}^{w} p_{ij}^{(k)}$$

And the pixel-wise mean μ_{ij} across all images is

$$\mu_{ij} = \frac{1}{n} \sum_{k=1}^{n} p_{ij}^{(k)}.$$

The general standard deviation σ and pixel-wise standard deviation σ_{ij} is defined similarly.

Which of the following preprocessing steps will not change the performance of a Nearest Neighbor classifier that uses L1 distance? Select all that apply. To clarify, both training and test examples are preprocessed in the same way.

1. Subtracting the mean μ ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu$.)
2. Subtracting the per pixel mean μ_{ij} ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu_{ij}$.)
3. Subtracting the mean μ and dividing by the standard deviation σ.
4. Subtracting the pixel-wise mean μ_{ij} and dividing by the pixel-wise standard deviation σ_{ij}.
5. Rotating the coordinate axes of the data, which means rotating all the images by the same angle. Empty regions in the image caused by rotation are padded with a same pixel value and no interpolation is performed.

Answer: 1, 2, 3

Explanation:

1. Subtracting the global mean does not change the L1 distances, since the same constant is subtracted from both training and test images.
2. Subtracting the pixel-wise mean also preserves L1 distances, because the same value is subtracted from each corresponding pixel in both images.
3. Dividing by the global standard deviation scales all L1 distances by the same positive constant. Therefore, their relative ordering remains unchanged, and the nearest neighbors stay the same.
4. Dividing by the pixel-wise standard deviation assigns different weights to different pixels. This can change the relative distances and the nearest-neighbor predictions.

5. Arbitrary rotations do not generally preserve L1 distances. However, rotations that only permute pixel positions without losing information, such as a 90 degree rotation, do preserve them.

[11]:
```
# Now lets speed up distance matrix computation by using partial vectorization
# with one loop. Implement the function compute_distances_one_loop and run the
# code below:
dists_one = classifier.compute_distances_one_loop(X_test)

# To ensure that our vectorized implementation is correct, we make sure that it
# agrees with the naive implementation. There are many ways to decide whether
# two matrices are similar; one of the simplest is the Frobenius norm. In case
# you haven't seen it before, the Frobenius norm of two matrices is the square
# root of the squared sum of differences of all elements; in other words, reshape
# the matrices into vectors and compute the Euclidean distance between them.
difference = np.linalg.norm(dists - dists_one, ord='fro')
print('One loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

```
One loop difference was: 0.000000
Good! The distance matrices are the same
```

[12]:
```
# Now implement the fully vectorized version inside compute_distances_no_loops
# and run the code
dists_two = classifier.compute_distances_no_loops(X_test)

# check that the distance matrix agrees with the one we computed before:
difference = np.linalg.norm(dists - dists_two, ord='fro')
print('No loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

```
No loop difference was: 0.000000
Good! The distance matrices are the same
```

[13]:
```
# Let's compare how fast the implementations are
def time_function(f, *args):
    """
    Call a function f with args and return the time (in seconds) that it took to execute.
    """
    import time
```

```
    tic = time.time()
    f(*args)
    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops, X_test)
print('Two loop version took %f seconds' % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop, X_test)
print('One loop version took %f seconds' % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops, X_test)
print('No loop version took %f seconds' % no_loop_time)

# You should see significantly faster performance with the fully vectorized␣
 ↪implementation!

# NOTE: depending on what machine you're using,
# you might not see a speedup when you go from two loops to one loop,
# and might even see a slow-down.
```

```
Two loop version took 37.336865 seconds
One loop version took 51.061829 seconds
No loop version took 0.960524 seconds
```

1.0.1 Cross-validation

We have implemented the k-Nearest Neighbor classifier but we set the value k = 5 arbitrarily. We will now determine the best value of this hyperparameter with cross-validation.

[14]:
```
num_folds = 5
k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100]

X_train_folds = []
y_train_folds = []

# Split training data into 5 folds
X_train_folds = np.array_split(X_train, num_folds)
y_train_folds = np.array_split(y_train, num_folds)


# A dictionary holding the accuracies for different values of k that we find
# when running cross-validation. After running cross-validation,
# k_to_accuracies[k] should be a list of length num_folds giving the different
# accuracy values that we found when using that value of k.
k_to_accuracies = {}
```

```
# Initialize accuracy lists for each k
for k in k_choices:
    k_to_accuracies[k] = []

# Perform 5-fold cross-validation
for fold in range(num_folds):

    # Select the validation fold
    X_val = X_train_folds[fold]
    y_val = y_train_folds[fold]

    # Combine the remaining folds for training
    X_tr = np.concatenate(
        [X_train_folds[j] for j in range(num_folds)
         if j != fold], axis=0
    )

    y_tr = np.concatenate(
        [y_train_folds[j] for j in range(num_folds)
         if j != fold], axis=0
    )

    # Train a new KNN classifier
    cv_classifier = knn_module.KNearestNeighbor()
    cv_classifier.train(X_tr, y_tr)

    # Compute distances once for each fold
    dists_cv = cv_classifier.compute_distances_no_loops(X_val)

    # Evaluate all k values
    for k in k_choices:
        y_val_pred = cv_classifier.predict_labels(dists_cv, k=k)

        accuracy = np.mean(y_val_pred == y_val)

        k_to_accuracies[k].append(accuracy)

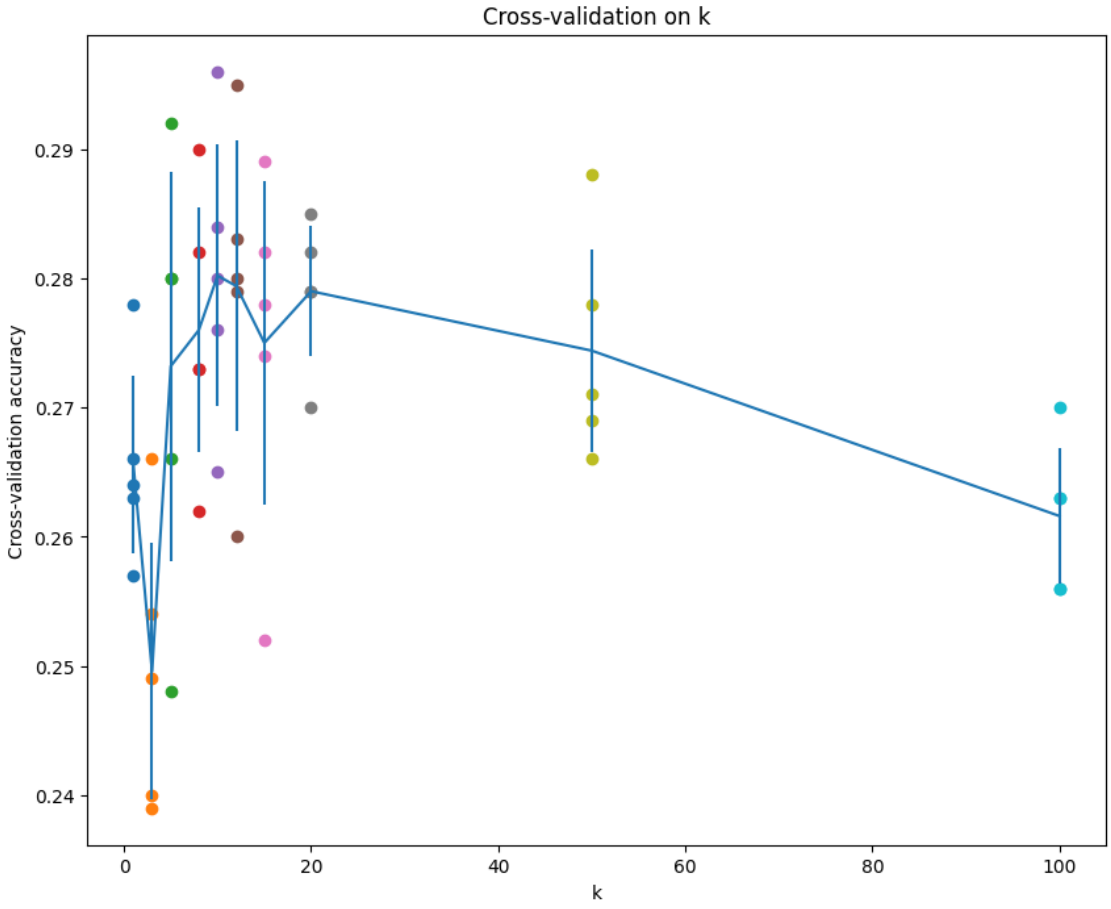

# Print out the computed accuracies
for k in sorted(k_to_accuracies):
    for accuracy in k_to_accuracies[k]:
        print('k = %d, accuracy = %f' % (k, accuracy))
```

```
k = 1, accuracy = 0.263000
k = 1, accuracy = 0.257000
k = 1, accuracy = 0.264000
k = 1, accuracy = 0.278000
```

```
k = 1, accuracy = 0.266000
k = 3, accuracy = 0.239000
k = 3, accuracy = 0.249000
k = 3, accuracy = 0.240000
k = 3, accuracy = 0.266000
k = 3, accuracy = 0.254000
k = 5, accuracy = 0.248000
k = 5, accuracy = 0.266000
k = 5, accuracy = 0.280000
k = 5, accuracy = 0.292000
k = 5, accuracy = 0.280000
k = 8, accuracy = 0.262000
k = 8, accuracy = 0.282000
k = 8, accuracy = 0.273000
k = 8, accuracy = 0.290000
k = 8, accuracy = 0.273000
k = 10, accuracy = 0.265000
k = 10, accuracy = 0.296000
k = 10, accuracy = 0.276000
k = 10, accuracy = 0.284000
k = 10, accuracy = 0.280000
k = 12, accuracy = 0.260000
k = 12, accuracy = 0.295000
k = 12, accuracy = 0.279000
k = 12, accuracy = 0.283000
k = 12, accuracy = 0.280000
k = 15, accuracy = 0.252000
k = 15, accuracy = 0.289000
k = 15, accuracy = 0.278000
k = 15, accuracy = 0.282000
k = 15, accuracy = 0.274000
k = 20, accuracy = 0.270000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.282000
k = 20, accuracy = 0.285000
k = 50, accuracy = 0.271000
k = 50, accuracy = 0.288000
k = 50, accuracy = 0.278000
k = 50, accuracy = 0.269000
k = 50, accuracy = 0.266000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.270000
k = 100, accuracy = 0.263000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.263000
```

[15]:
```
# plot the raw observations
for k in k_choices:
    accuracies = k_to_accuracies[k]
    plt.scatter([k] * len(accuracies), accuracies)

# plot the trend line with error bars that correspond to standard deviation
accuracies_mean = np.array([np.mean(v) for k,v in sorted(k_to_accuracies.
 ↪items())])
accuracies_std = np.array([np.std(v) for k,v in sorted(k_to_accuracies.
 ↪items())])
plt.errorbar(k_choices, accuracies_mean, yerr=accuracies_std)
plt.title('Cross-validation on k')
plt.xlabel('k')
plt.ylabel('Cross-validation accuracy')
plt.show()
```



[16]:
```
# Based on the cross-validation results above, choose the best value for k,
# retrain the classifier using all the training data, and test it on the test
```

```
# data. You should be able to get above 28% accuracy on the test data.
best_k = 10

classifier = knn_module.KNearestNeighbor()
classifier.train(X_train, y_train)
y_test_pred = classifier.predict(X_test, k=best_k)

# Compute and display the accuracy
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 141 / 500 correct => accuracy: 0.282000
```

Inline Question 3

Which of the following statements about k-Nearest Neighbor (k-NN) are true in a classification setting, and for all k? Select all that apply. 1. The decision boundary of the k-NN classifier is linear. 2. The training error of a 1-NN will always be lower than or equal to that of 5-NN. 3. The test error of a 1-NN will always be lower than that of a 5-NN. 4. The time needed to classify a test example with the k-NN classifier grows with the size of the training set. 5. None of the above.

Answer: 2, 4

Explanation:

1. False. The decision boundary of k-NN is generally nonlinear and depends on the distribution of the training data.
2. True. Assuming each training example is its own nearest neighbor and there are no conflicting duplicate samples, 1-NN achieves zero training error. Therefore, its training error is less than or equal to that of 5-NN.
3. False. The test error depends on the dataset and the choice of k. There is no guarantee that 1-NN performs better than 5-NN.
4. True. In the standard brute-force k-NN implementation, classifying a test example requires computing distances to all training examples. Therefore, the prediction time increases with the size of the training set.
5. False, since statements 2 and 4 are correct.

softmax

November 10, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# assignment folder, e.g. 'icv83551/assignments/assignment1/'
FOLDERNAME = 'icv83551/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/icv83551/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
/content/drive/My Drive/icv83551/assignments/assignment1/icv83551/datasets
/content/drive/My Drive/icv83551/assignments/assignment1
```

1 Softmax Classifier exercise

*Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission.

In this exercise you will:

- Implement a fully-vectorized **loss function** for the Softmax classifier.
- Implement the fully-vectorized expression for its **analytic gradient**
- **Check your implementation** using numerical gradient
- Use a validation set to **tune the learning rate and regularization** strength
- **Optimize** the loss function with **SGD**
- **Visualize** the final learned weights

[2]:
```
# Run some setup code for this notebook.
import random
import numpy as np
from icv83551.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

#%load_ext autoreload
#%autoreload 2
```

1.1 CIFAR-10 Data Loading and Preprocessing

[3]:
```
# Load the raw CIFAR-10 data.
cifar10_dir = 'icv83551/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause␣
 ↪memory issue)
try:
   del X_train, y_train
   del X_test, y_test
   print('Clear previously loaded data.')
except:
   pass

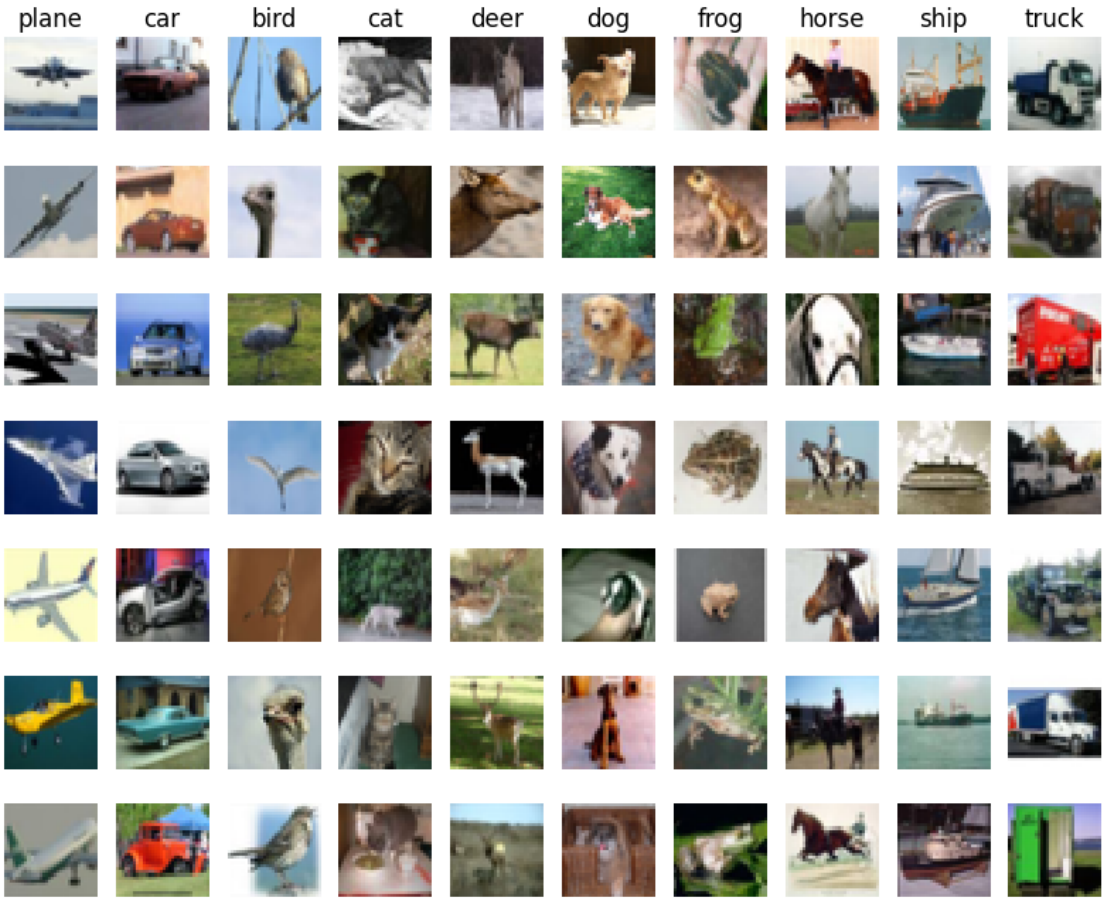
X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
```

[4]:
```
# Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']
```

```
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



[5]:
```
# Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
```

```
num_validation = 1000
num_test = 1000
num_dev = 500

# Our validation set will be num_validation points from the original
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
y_train = y_train[mask]

# We will also make a development set, which is a small subset of
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]

# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Train data shape:  (49000, 32, 32, 3)
Train labels shape:  (49000,)
Validation data shape:  (1000, 32, 32, 3)
Validation labels shape:  (1000,)
Test data shape:  (1000, 32, 32, 3)
Test labels shape:  (1000,)
```

[6]:
```
# Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))
```

```
# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)
```

```
Training data shape:  (49000, 3072)
Validation data shape:  (1000, 3072)
Test data shape:  (1000, 3072)
dev data shape:  (500, 3072)
```

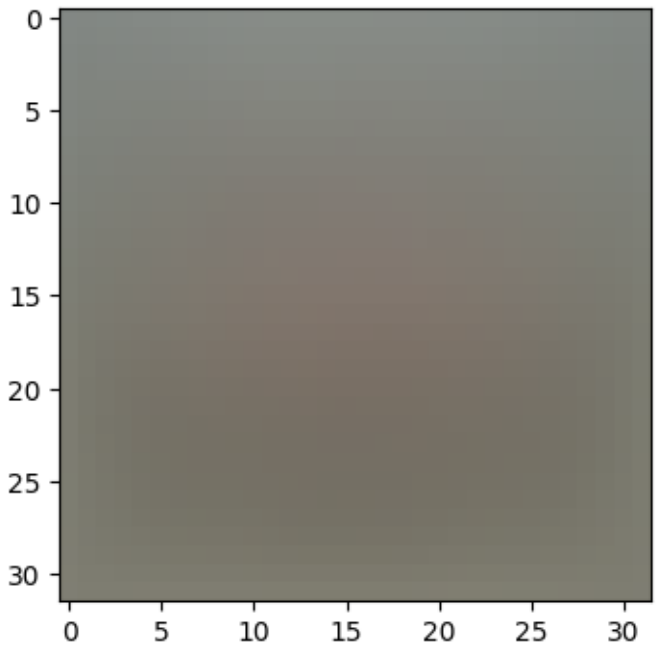
[7]:
```
# Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)
print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the mean␣
 ↪image
plt.show()

# second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image

# third: append the bias dimension of ones (i.e. bias trick) so that our␣
 ↪classifier
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)
```

```
[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]
```




```
(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)
```

1.2 Softmax Classifier

Your code for this section will all be written inside `icv83551/classifiers/softmax.py`.

As you can see, we have prefilled the function `softmax_loss_naive` which uses for loops to evaluate the softmax loss function.

[8]:
```
# Evaluate the naive implementation of the loss we provided for you:
from icv83551.classifiers.softmax import softmax_loss_naive
import time

# generate a random Softmax classifier weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001

loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
print('loss: %f' % (loss, ))

# As a rough sanity check, our loss should be something close to -log(0.1).
print('loss: %f' % loss)
print('sanity check: %f' % (-np.log(0.1)))
```

```
loss: 2.370051
loss: 2.370051
sanity check: 2.302585
```

Inline Question 1

Why do we expect our loss to be close to -log(0.1)? Explain briefly.

Answer:

Since the weights are initialized with small random values, the classifier initially produces approximately equal scores for all 10 classes.

Therefore, the Softmax probabilities are approximately uniform, assigning a probability of 0.1 to each class.

Since the cross-entropy loss is the negative logarithm of the probability assigned to the correct class, we expect:

L = -log(0.1) = log(10) 2.302585.

Small deviations may occur due to random initialization and regularization.

The `grad` returned from the function above is right now all zero. Derive and implement the gradient for the softmax loss function and implement it inline inside the function `softmax_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you:

[9]:
```
# Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensions, and
# compare them with your analytically computed gradient. The numbers should␣
 ↪match
# almost exactly along all dimensions.
from icv83551.gradient_check import grad_check_sparse
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)
```

```
numerical: -1.735637 analytic: -1.735637, relative error: 2.262308e-09
numerical: -3.398134 analytic: -3.398134, relative error: 8.202292e-09
numerical: 1.754937 analytic: 1.754938, relative error: 4.701816e-09
numerical: -0.809591 analytic: -0.809591, relative error: 2.402182e-08
numerical: 0.606612 analytic: 0.606611, relative error: 1.017210e-07
numerical: 0.564246 analytic: 0.564246, relative error: 5.532337e-08
numerical: 0.506620 analytic: 0.506620, relative error: 1.612561e-07
```

```
numerical: 1.147228 analytic: 1.147228, relative error: 4.187157e-08
numerical: 0.647844 analytic: 0.647844, relative error: 3.666609e-08
numerical: -2.518258 analytic: -2.518258, relative error: 1.642718e-08
numerical: -1.068053 analytic: -1.068053, relative error: 4.667622e-08
numerical: -0.599801 analytic: -0.599801, relative error: 1.069321e-08
numerical: 0.800668 analytic: 0.800668, relative error: 1.186702e-07
numerical: -0.925018 analytic: -0.925018, relative error: 3.272391e-08
numerical: -0.446022 analytic: -0.446022, relative error: 1.238607e-09
numerical: -5.923549 analytic: -5.923550, relative error: 5.125834e-09
numerical: -3.190610 analytic: -3.190610, relative error: 1.857416e-08
numerical: -1.253990 analytic: -1.253990, relative error: 1.239852e-08
numerical: 5.799033 analytic: 5.799033, relative error: 2.752781e-08
numerical: -0.423912 analytic: -0.423912, relative error: 2.254289e-07
```

Inline Question 2

Although gradcheck is reliable softmax loss, it is possible that for SVM loss, once in a while, a dimension in the gradcheck will not match exactly. What could such a discrepancy be caused by? Is it a reason for concern? What is a simple example in one dimension where a svm loss gradient check could fail? How would change the margin affect of the frequency of this happening?

Note that SVM loss for a sample (x_i, y_i) is defined as:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$$

Where j iterates over all classes except the correct class y_i and s_j denotes the classifier score for j^{th} class. Δ is a scalar margin. *Hint: the SVM loss function is not strictly speaking differentiable.*

Answer:

The discrepancy may occur because the SVM hinge loss is not differentiable at points where the margin constraint is exactly satisfied, i.e., when

$s_j - s_{y_i} + \Delta = 0.$

Near such points, a numerical gradient check may cross the non-differentiable boundary and produce a value different from the analytical gradient.

This is not necessarily a reason for concern, as occasional discrepancies near these points are expected.

For example, consider the one-dimensional function

$f(w) = \max(0, w + 1).$

At $w = -1$, the function is not differentiable. The centered numerical gradient gives 0.5, while the analytical implementation may use a subgradient of 0 or 1.

Changing the margin Δ shifts the location of these non-differentiable boundaries. The frequency of discrepancies depends on the distribution of the scores. For scores concentrated near zero, decreasing the margin toward zero may increase the frequency of such discrepancies.

[10]:
```
# Next implement the function softmax_loss_vectorized; for now only compute the␣
↪loss;
```

```
# we will implement the gradient in a moment.
tic = time.time()
loss_naive, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))

from icv83551.classifiers.softmax import softmax_loss_vectorized
tic = time.time()
loss_vectorized, _ = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

# The losses should match but your vectorized implementation should be much␣
 ↪faster.
print('difference: %f' % (loss_naive - loss_vectorized))
```

```
Naive loss: 2.370051e+00 computed in 0.061791s
Vectorized loss: 2.370051e+00 computed in 0.012485s
difference: -0.000000
```

[11]:
```
# Complete the implementation of softmax_loss_vectorized, and compute the␣
 ↪gradient
# of the loss function in a vectorized way.

# The naive implementation and the vectorized implementation should match, but
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))

tic = time.time()
_, grad_vectorized = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values computed
# by the two implementations. The gradient on the other hand is a matrix, so
# we use the Frobenius norm to compare them.
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)
```

```
Naive loss and gradient: computed in 0.089092s
Vectorized loss and gradient: computed in 0.011167s
difference: 0.000000
```

1.2.1 Stochastic Gradient Descent

We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss. Your code for this part will be written inside `icv83551/classifiers/linear_classifier.py`.

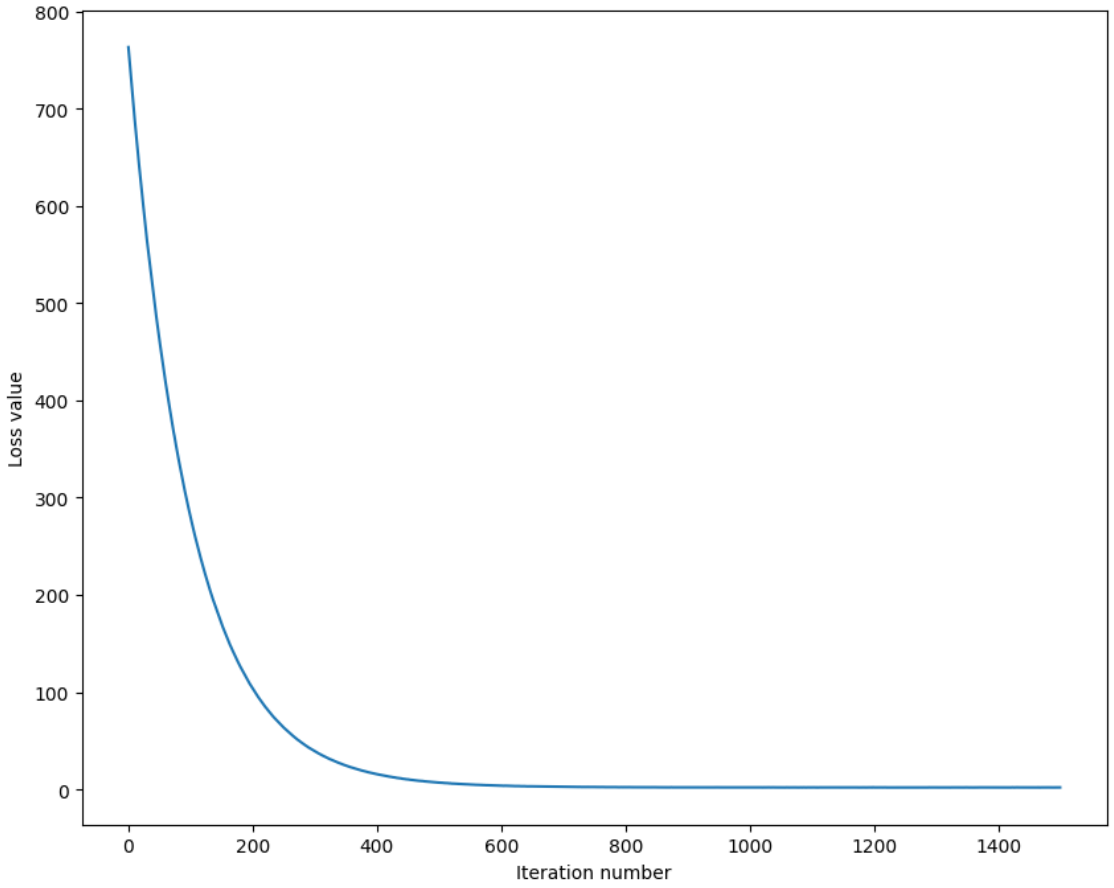
[15]:
```
# In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.
import importlib
import icv83551.classifiers.linear_classifier as linear_module

importlib.reload(linear_module)
Softmax = linear_module.Softmax

softmax = Softmax()
tic = time.time()
loss_hist = softmax.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                      num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))
```

```
iteration 0 / 1500: loss 763.123451
iteration 100 / 1500: loss 280.046847
iteration 200 / 1500: loss 103.701300
iteration 300 / 1500: loss 39.231133
iteration 400 / 1500: loss 15.718772
iteration 500 / 1500: loss 7.069307
iteration 600 / 1500: loss 3.922389
iteration 700 / 1500: loss 2.740163
iteration 800 / 1500: loss 2.303055
iteration 900 / 1500: loss 2.137157
iteration 1000 / 1500: loss 2.139514
iteration 1100 / 1500: loss 2.077574
iteration 1200 / 1500: loss 2.079028
iteration 1300 / 1500: loss 2.094810
iteration 1400 / 1500: loss 2.146529
That took 12.711854s
```

[16]:
```
# A useful debugging strategy is to plot the loss as a function of
# iteration number:
plt.plot(loss_hist)
plt.xlabel('Iteration number')
plt.ylabel('Loss value')
plt.show()
```



[17]:
```
# Write the LinearClassifier.predict function and evaluate the performance on
# both the training and validation set
# You should get validation accuracy of about 0.34 (> 0.33).
y_train_pred = softmax.predict(X_train)
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = softmax.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

```
training accuracy: 0.333204
validation accuracy: 0.345000
```

[18]:
```
# Save the trained model for autograder.
softmax.save("softmax.npy")
```

```
softmax.npy saved.
```

[21]:
```
# Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able to
# get a classification accuracy of about 0.365 (> 0.36) on the validation set.
```

```
# Note: you may see runtime/overflow warnings during hyper-parameter search.
# This may be caused by extreme values, and is not a bug.

# results is dictionary mapping tuples of the form
# (learning_rate, regularization_strength) to tuples of the form
# (training_accuracy, validation_accuracy). The accuracy is simply the fraction
# of data points that are correctly classified.
results = {}
best_val = -1   # The highest validation accuracy that we have seen so far.
best_softmax = None # The Softmax object that achieved the highest validation␣
 ↪rate.


# Provided as a reference. You may or may not want to change these␣
 ↪hyperparameters
learning_rates = [1e-7, 2e-7, 5e-7]
regularization_strengths = [1e3, 1e4, 2.5e4]


# Make sure we use the updated classifier implementation
import importlib
import icv83551.classifiers.linear_classifier as linear_module

importlib.reload(linear_module)
Softmax = linear_module.Softmax

# Search for the best hyperparameters
for lr in learning_rates:
    for reg in regularization_strengths:

        # Create a new Softmax classifier
        model = Softmax()

        # Train the classifier
        model.train(
            X_train,
            y_train,
            learning_rate=lr,
            reg=reg,
            num_iters=1500,
            batch_size=200,
            verbose=False
        )

        # Compute training accuracy
        y_train_pred = model.predict(X_train)
```

```
        train_accuracy = np.mean(y_train_pred == y_train)

        # Compute validation accuracy
        y_val_pred = model.predict(X_val)
        val_accuracy = np.mean(y_val_pred == y_val)

        # Store the results
        results[(lr, reg)] = (train_accuracy, val_accuracy)

        # Update the best classifier
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = model


# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
                lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved during cross-validation: %f' %␣
 ↪best_val)
```

```
lr 1.000000e-07 reg 1.000000e+03 train accuracy: 0.265469 val accuracy: 0.260000
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.354388 val accuracy: 0.371000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.326673 val accuracy: 0.338000
lr 2.000000e-07 reg 1.000000e+03 train accuracy: 0.313000 val accuracy: 0.313000
lr 2.000000e-07 reg 1.000000e+04 train accuracy: 0.357306 val accuracy: 0.372000
lr 2.000000e-07 reg 2.500000e+04 train accuracy: 0.333837 val accuracy: 0.347000
lr 5.000000e-07 reg 1.000000e+03 train accuracy: 0.387776 val accuracy: 0.395000
lr 5.000000e-07 reg 1.000000e+04 train accuracy: 0.353163 val accuracy: 0.367000
lr 5.000000e-07 reg 2.500000e+04 train accuracy: 0.322918 val accuracy: 0.337000
best validation accuracy achieved during cross-validation: 0.395000
```

[22]:
```
# Visualize the cross-validation results
import math
import pdb

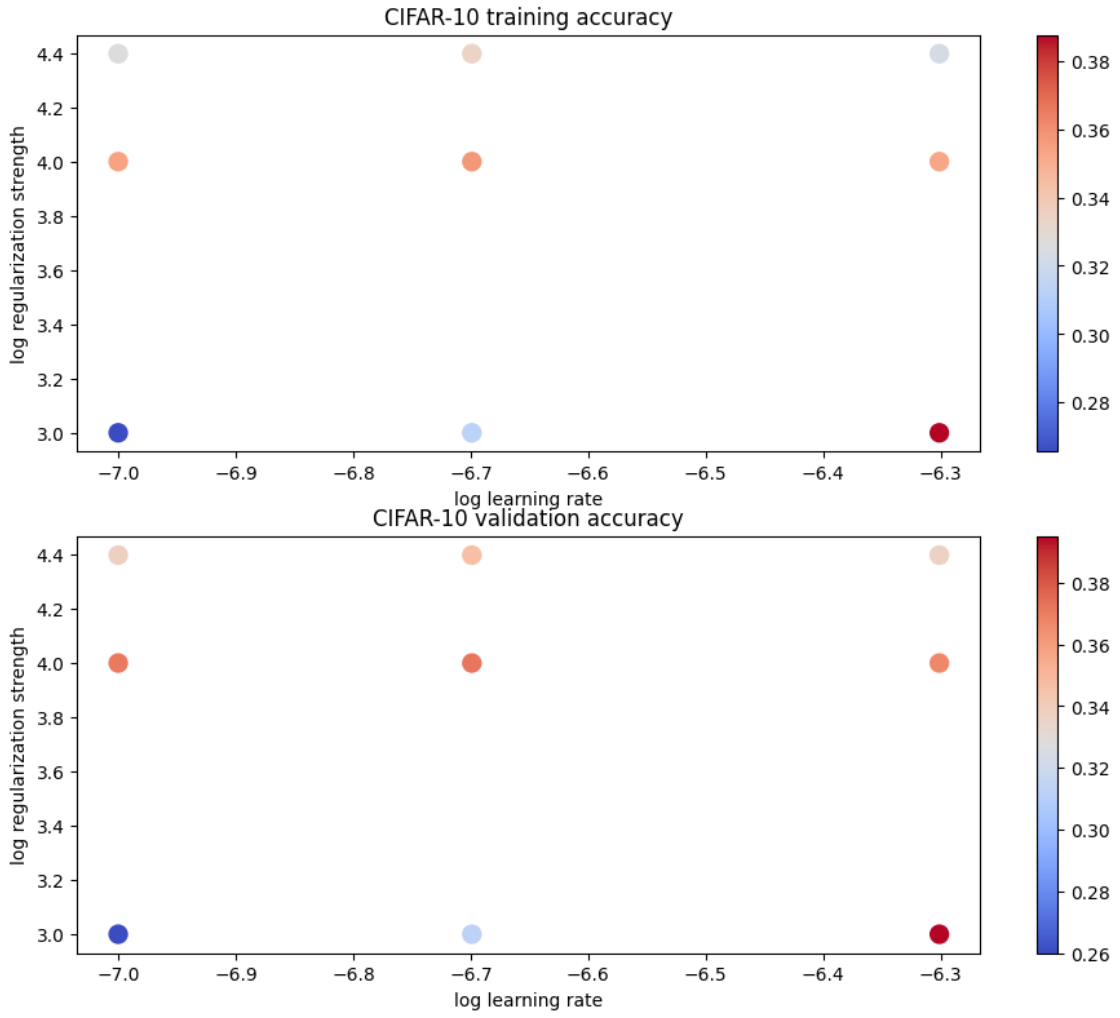
# pdb.set_trace()

x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
```

```
plt.subplot(2, 1, 1)
plt.tight_layout(pad=3)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 validation accuracy')
plt.show()
```



[23]:
```
# Evaluate the best softmax on test set
y_test_pred = best_softmax.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('Softmax classifier on raw pixels final test set accuracy: %f' %␣
 ↪test_accuracy)
```

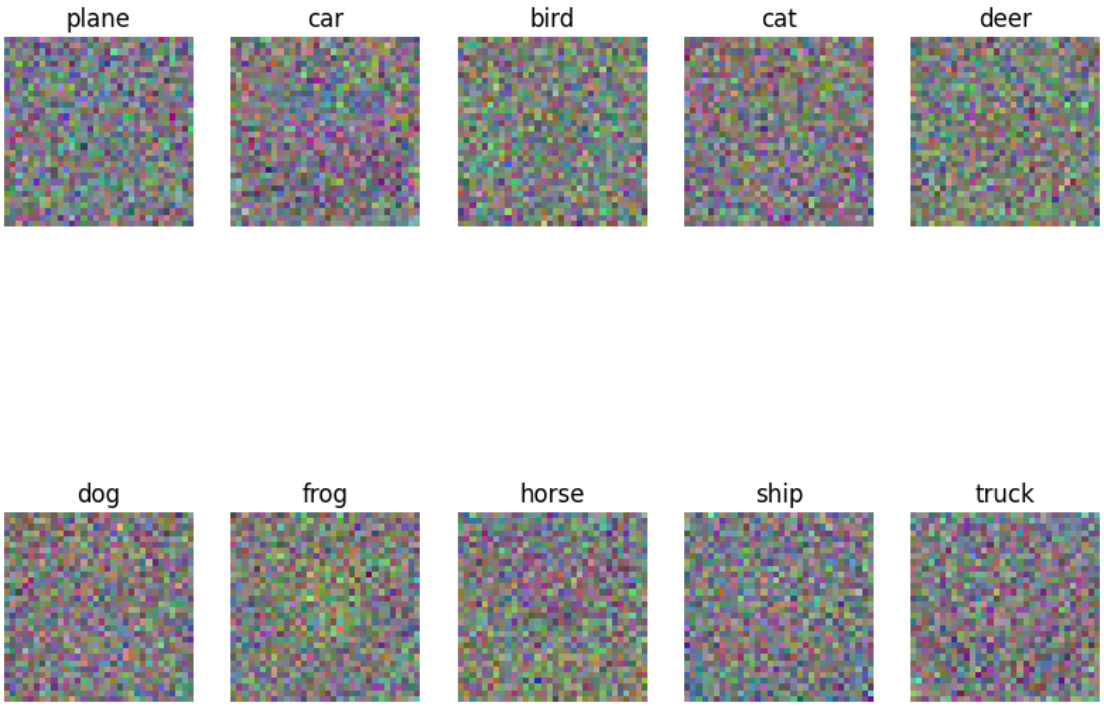
```
Softmax classifier on raw pixels final test set accuracy: 0.373000
```

[24]:
```
# Save best softmax model
best_softmax.save("best_softmax.npy")
```

```
best_softmax.npy saved.
```

[25]:
```
# Visualize the learned weights for each class.
# Depending on your choice of learning rate and regularization strength, these␣
 ↪may
# or may not be nice to look at.
w = best_softmax.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)
w_min, w_max = np.min(w), np.max(w)
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
    plt.axis('off')
    plt.title(classes[i])
```



Inline question 3

Describe what your visualized Softmax classifier weights look like, and offer a brief explanation for why they look the way they do.

Answer:

The visualized Softmax weights appear mostly noisy and colorful, without clearly recognizable objects. Some weak class-specific color and spatial patterns may be visible.

This is because the Softmax classifier is a linear model that learns a weight for each pixel and class. It operates directly on raw pixel values and cannot learn complex spatial features or object shapes.

Therefore, the weights mainly capture low-level color and pixel patterns rather than clear representations of the objects.

Inline Question 4 - *True or False*

Suppose the overall training loss is defined as the sum of the per-datapoint loss over all training examples. It is possible to add a new datapoint to a training set that would change the softmax loss, but leave the SVM loss unchanged.

Answer: True.

Explanation:

The SVM hinge loss becomes zero when the correct class score exceeds all incorrect class scores by at least the margin. Therefore, adding a new training example that satisfies this condition does not change the total SVM loss.

However, the Softmax cross-entropy loss is strictly positive for finite scores, since the probability assigned to the correct class is less than 1.

For example, with two classes, a margin of 1, and scores of 2 for the correct class and 0 for the incorrect class, the SVM loss is zero, while the Softmax loss is approximately 0.127.

Thus, adding this example changes the total Softmax loss but leaves the total SVM loss unchanged.

two_layer_net

November 10, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'icv83551/assignments/assignment1/'
FOLDERNAME = 'icv83551/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/icv83551/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
/content/drive/My Drive/icv83551/assignments/assignment1/icv83551/datasets
/content/drive/My Drive/icv83551/assignments/assignment1
```

1 Fully-Connected Neural Nets

In this exercise we will implement fully-connected networks using a modular approach. For each layer we will implement a `forward` and a `backward` function. The `forward` function will receive inputs, weights, and other parameters and will return both an output and a `cache` object storing data needed for the backward pass, like this:

```
def layer_forward(x, w):
  """ Receive inputs x and weights w """
  # Do some computations ...
  z = # ... some intermediate value
  # Do some more computations ...
```

```
  out = # the output

  cache = (x, w, z, out) # Values we need to compute gradients

  return out, cache
```

The backward pass will receive upstream derivatives and the `cache` object, and will return gradients with respect to the inputs and weights, like this:

```
def layer_backward(dout, cache):
  """
  Receive dout (derivative of loss with respect to outputs) and cache,
  and compute derivative with respect to inputs.
  """
  # Unpack cache values
  x, w, z, out = cache

  # Use values in cache to compute derivatives
  dx = # Derivative of loss with respect to x
  dw = # Derivative of loss with respect to w

  return dx, dw
```

After implementing a bunch of layers this way, we will be able to easily combine them to build classifiers with different architectures.

[2]:
```
# As usual, a bit of setup
from __future__ import print_function
import time
import numpy as np
import matplotlib.pyplot as plt
from icv83551.classifiers.fc_net import *
from icv83551.data_utils import get_CIFAR10_data
from icv83551.gradient_check import eval_numerical_gradient,␣
 ↪eval_numerical_gradient_array
from icv83551.solver import Solver

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

#%load_ext autoreload
#%autoreload 2

def rel_error(x, y):
  """ returns relative error """
  return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

[3]:
```
# Load the (preprocessed) CIFAR10 data.

data = get_CIFAR10_data()
for k, v in list(data.items()):
  print(('%s: ' % k, v.shape))
```

```
('X_train: ', (49000, 3, 32, 32))
('y_train: ', (49000,))
('X_val: ', (1000, 3, 32, 32))
('y_val: ', (1000,))
('X_test: ', (1000, 3, 32, 32))
('y_test: ', (1000,))
```

2 Affine layer: forward

Open the file `icv83551/layers.py` and implement the `affine_forward` function.

Once you are done you can test your implementaion by running the following:

[4]:
```
# Test the affine_forward function

num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_shape)
w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape),␣
 ↪output_dim)
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,   3.77273342]])

# Compare your output with ours. The error should be around e-9 or less.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing affine_forward function:
difference:  9.769849468192957e-10
```

3 Affine layer: backward

Now implement the `affine_backward` function and test your implementation using numeric gradient checking.

[5]:
```
# Test the affine_backward function
np.random.seed(231)
x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[0], x,␣
 ↪dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[0], w,␣
 ↪dout)
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[0], b,␣
 ↪dout)

_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

# The error should be around e-10 or less
print('Testing affine_backward function:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_backward function:
dx error:  5.399100368651805e-11
dw error:  9.904211865398145e-11
db error:  2.4122867568119087e-11
```

4 ReLU activation: forward

Implement the forward pass for the ReLU activation function in the `relu_forward` function and test your implementation using the following:

[6]:
```
# Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.,        ],
                        [ 0.,          0.,          0.04545455,  0.13636364,],
                        [ 0.22727273,  0.31818182,  0.40909091,  0.5,       ]])

# Compare your output with ours. The error should be on the order of e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing relu_forward function:
difference:  4.999999798022158e-08
```

5 ReLU activation: backward

Now implement the backward pass for the ReLU activation function in the `relu_backward` function and test your implementation using numeric gradient checking:

[7]:
```
np.random.seed(231)
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be on the order of e-12
print('Testing relu_backward function:')
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing relu_backward function:
dx error:  3.2756349136310288e-12
```

5.1 Inline Question 1:

We've only asked you to implement ReLU, but there are a number of different activation functions that one could use in neural networks, each with its pros and cons. In particular, an issue commonly seen with activation functions is getting zero (or close to zero) gradient flow during backpropagation. Which of the following activation functions have this problem? If you consider these functions in the one dimensional case, what types of input would lead to this behaviour? 1. Sigmoid 2. ReLU 3. Leaky ReLU

Answer : 1. Sigmoid: Suffers from vanishing gradients for very large positive or negative inputs, where the function saturates and its derivative approaches zero.

2. ReLU: Has a zero gradient for negative inputs, which can cause neurons to stop learning (the dying ReLU problem).
3. Leaky ReLU: Avoids exactly zero gradients for negative inputs by using a small positive slope alpha. However, the gradient can still be close to zero if alpha is very small.*

6 "Sandwich" layers

There are some common patterns of layers that are frequently used in neural nets. For example, affine layers are frequently followed by a ReLU nonlinearity. To make these common patterns easy, we define several convenience layers in the file `icv83551/layer_utils.py`.

For now take a look at the `affine_relu_forward` and `affine_relu_backward` functions, and run the following to numerically gradient check the backward pass:

[8]:
```
from icv83551.layer_utils import affine_relu_forward, affine_relu_backward
np.random.seed(231)
```

```
x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w,␣
 ↪b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w,␣
 ↪b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w,␣
 ↪b)[0], b, dout)

# Relative error should be around e-10 or less
print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_relu_forward and affine_relu_backward:
dx error:  2.299579177309368e-11
dw error:  8.162011105764925e-11
db error:  7.826724021458994e-12
```

7 Loss layers: Softmax

Now implement the loss and gradient for softmax in the `softmax_loss` function in `icv83551/layers.py`. These should be similar to what you implemented in `icv83551/classifiers/softmax.py`. Other loss functions (e.g. `svm_loss`) can also be implemented in a modular way, however, it is not required for this assignment.

You can make sure that the implementations are correct by running the following:

[9]:
```
np.random.seed(231)
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)


dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x,␣
 ↪verbose=False)
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be close to 2.3 and dx error should␣
 ↪be around e-8
```

```
print('\nTesting softmax_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing softmax_loss:
loss:  2.302545844500738
dx error:  9.384673161989355e-09
```

8 Two-layer network

Open the file `icv83551/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class. Read through it to make sure you understand the API. You can run the cell below to test your implementation.

[10]:
```
np.random.seed(231)
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-3
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C, weight_scale=std)

print('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
  [[11.53165108,  12.2917344,   13.05181771,  13.81190102,  14.57198434, 15.
 ↪33206765,  16.09215096],
   [12.05769098,  12.74614105,  13.43459113,  14.1230412,   14.81149128, 15.
 ↪49994135,  16.18839143],
   [12.58373087,  13.20054771,  13.81736455,  14.43418138,  15.05099822, 15.
 ↪66781506,  16.2846319 ]])
```

```
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss'

model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization loss'

# Errors should be around e-7 or less
for reg in [0.0, 0.7]:
  print('Running numeric gradient check with reg = ', reg)
  model.reg = reg
  loss, grads = model.loss(X, y)

  for name in sorted(grads):
    f = lambda _: model.loss(X, y)[0]
    grad_num = eval_numerical_gradient(f, model.params[name], verbose=False)
    print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))
```

```
Testing initialization …
Testing test-time forward pass …
Testing training loss (no regularization)
Running numeric gradient check with reg =  0.0
W1 relative error: 1.83e-08
W2 relative error: 3.12e-10
b1 relative error: 9.83e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg =  0.7
W1 relative error: 2.53e-07
W2 relative error: 2.85e-08
b1 relative error: 1.56e-08
b2 relative error: 7.76e-10
```

9 Solver

Open the file `icv83551/solver.py` and read through it to familiarize yourself with the API. After doing so, use a `Solver` instance to train a `TwoLayerNet` that achieves about `36%` accuracy on the validation set.

[13]:
```
input_size = 32 * 32 * 3
hidden_size = 50
```

```
num_classes = 10
model = TwoLayerNet(input_size, hidden_size, num_classes)
solver = None


# Initialize the Solver
solver = Solver(
    model,
    data,
    update_rule='sgd',
    optim_config={
        'learning_rate': 1e-3
    },
    lr_decay=0.95,
    num_epochs=10,
    batch_size=100,
    print_every=100,
    verbose=True
)

# Train the model
solver.train()

# Display best validation accuracy
print("Best validation accuracy:",
      max(solver.val_acc_history))
```

```
(Iteration 1 / 4900) loss: 2.302908
(Epoch 0 / 10) train acc: 0.091000; val_acc: 0.096000
(Iteration 101 / 4900) loss: 1.838519
(Iteration 201 / 4900) loss: 1.876793
(Iteration 301 / 4900) loss: 1.741901
(Iteration 401 / 4900) loss: 1.454628
(Epoch 1 / 10) train acc: 0.403000; val_acc: 0.416000
(Iteration 501 / 4900) loss: 1.573056
(Iteration 601 / 4900) loss: 1.710478
(Iteration 701 / 4900) loss: 1.710300
(Iteration 801 / 4900) loss: 1.676016
(Iteration 901 / 4900) loss: 1.433272
(Epoch 2 / 10) train acc: 0.484000; val_acc: 0.460000
(Iteration 1001 / 4900) loss: 1.480785
(Iteration 1101 / 4900) loss: 1.389022
(Iteration 1201 / 4900) loss: 1.660187
(Iteration 1301 / 4900) loss: 1.588942
(Iteration 1401 / 4900) loss: 1.240009
(Epoch 3 / 10) train acc: 0.485000; val_acc: 0.445000
(Iteration 1501 / 4900) loss: 1.416385
(Iteration 1601 / 4900) loss: 1.345758
```

```
(Iteration 1701 / 4900) loss: 1.309953
(Iteration 1801 / 4900) loss: 1.411325
(Iteration 1901 / 4900) loss: 1.482173
(Epoch 4 / 10) train acc: 0.505000; val_acc: 0.448000
(Iteration 2001 / 4900) loss: 1.421636
(Iteration 2101 / 4900) loss: 1.443772
(Iteration 2201 / 4900) loss: 1.388593
(Iteration 2301 / 4900) loss: 1.364996
(Iteration 2401 / 4900) loss: 1.405091
(Epoch 5 / 10) train acc: 0.503000; val_acc: 0.462000
(Iteration 2501 / 4900) loss: 1.444686
(Iteration 2601 / 4900) loss: 1.346272
(Iteration 2701 / 4900) loss: 1.228322
(Iteration 2801 / 4900) loss: 1.314194
(Iteration 2901 / 4900) loss: 1.307166
(Epoch 6 / 10) train acc: 0.522000; val_acc: 0.475000
(Iteration 3001 / 4900) loss: 1.307035
(Iteration 3101 / 4900) loss: 1.388436
(Iteration 3201 / 4900) loss: 1.299453
(Iteration 3301 / 4900) loss: 1.321536
(Iteration 3401 / 4900) loss: 1.466846
(Epoch 7 / 10) train acc: 0.532000; val_acc: 0.488000
(Iteration 3501 / 4900) loss: 1.300858
(Iteration 3601 / 4900) loss: 1.196168
(Iteration 3701 / 4900) loss: 1.377785
(Iteration 3801 / 4900) loss: 1.209821
(Iteration 3901 / 4900) loss: 1.205117
(Epoch 8 / 10) train acc: 0.529000; val_acc: 0.449000
(Iteration 4001 / 4900) loss: 1.311643
(Iteration 4101 / 4900) loss: 1.429700
(Iteration 4201 / 4900) loss: 1.231697
(Iteration 4301 / 4900) loss: 1.147972
(Iteration 4401 / 4900) loss: 1.472548
(Epoch 9 / 10) train acc: 0.561000; val_acc: 0.485000
(Iteration 4501 / 4900) loss: 1.169045
(Iteration 4601 / 4900) loss: 1.444726
(Iteration 4701 / 4900) loss: 1.034735
(Iteration 4801 / 4900) loss: 1.152693
(Epoch 10 / 10) train acc: 0.576000; val_acc: 0.487000
Best validation accuracy: 0.488
```

10 Debug the training

With the default parameters we provided above, you should get a validation accuracy of about 0.36 on the validation set. This isn't very good.

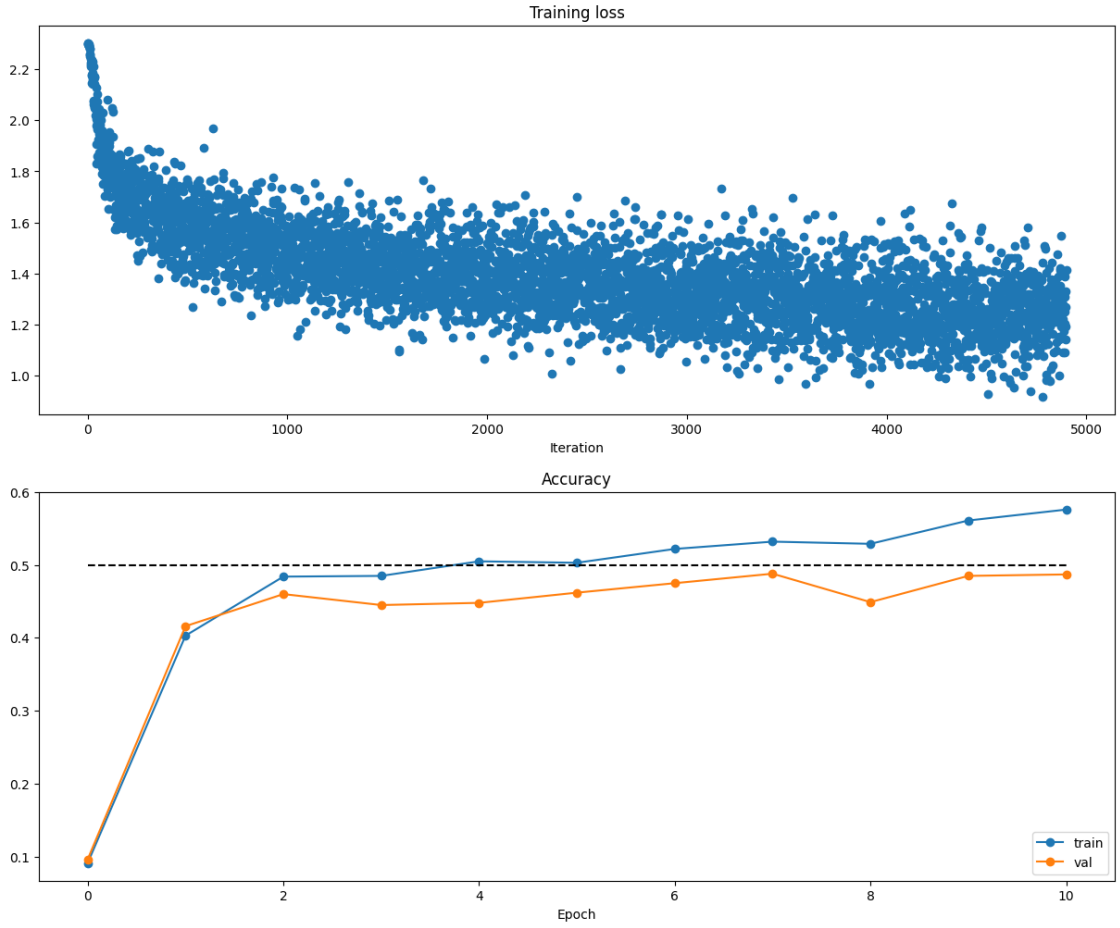
One strategy for getting insight into what's wrong is to plot the loss function and the accuracies on the training and validation sets during optimization.

Another strategy is to visualize the weights that were learned in the first layer of the network. In most neural networks trained on visual data, the first layer weights typically show some visible structure when visualized.

[14]:
```
# Run this cell to visualize training loss and train / val accuracy

plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```

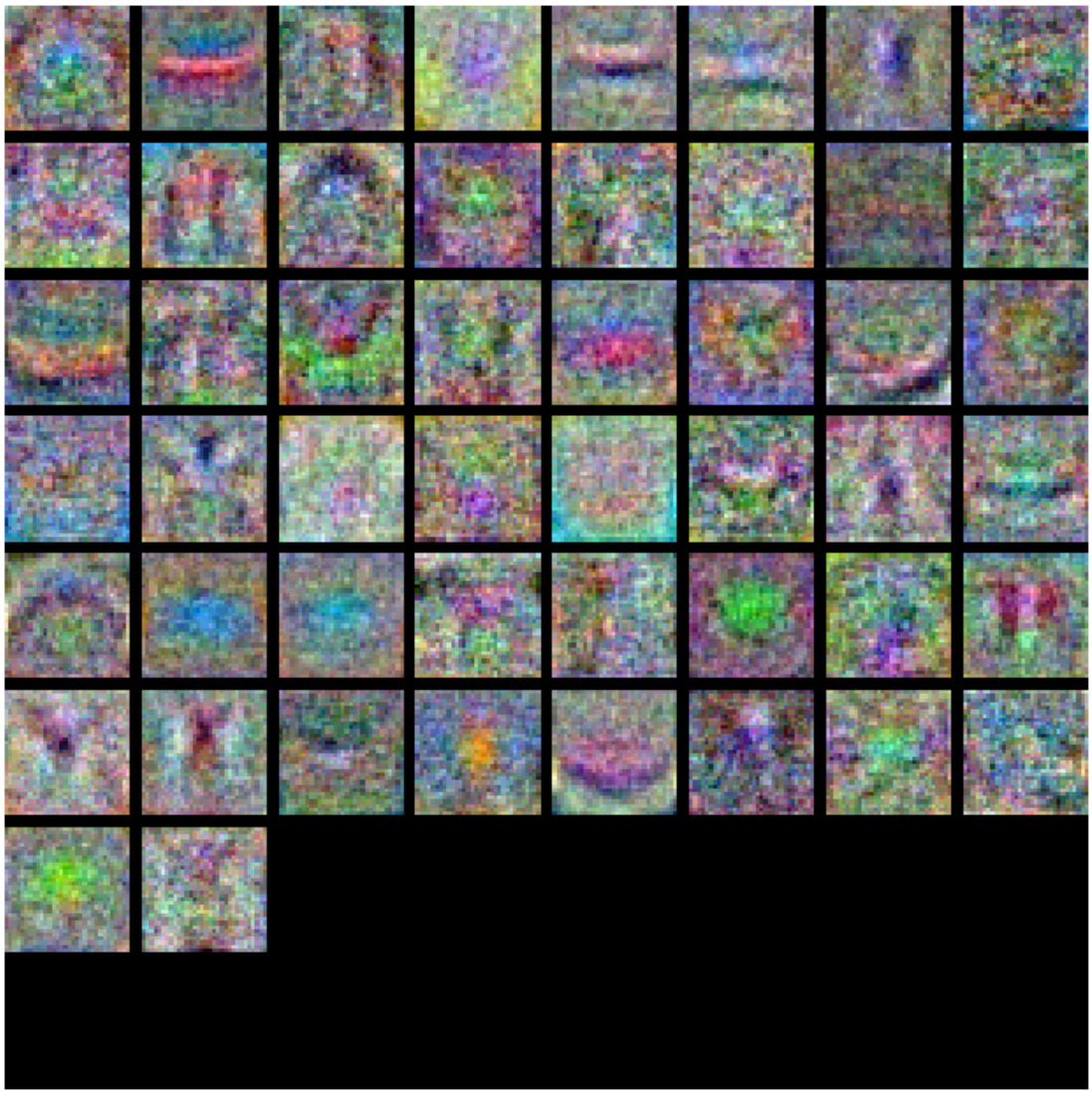


[15]:
```
from icv83551.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(3, 32, 32, -1).transpose(3, 1, 2, 0)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(model)
```



11 Tune your hyperparameters

What's wrong?. Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

Tuning. Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, numer of training epochs, and regularization strength. You might also consider tuning the learning rate decay, but you should be able to get good performance using the default value.

Approximate results. You should be aim to achieve a classification accuracy of greater than 48% on the validation set. Our best network gets over 52% on the validation set.

Experiment: You goal in this exercise is to get as good of a result on CIFAR-10 as you can (52% could serve as a reference), with a fully-connected Neural Network. Feel free implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

[17]:
```
best_model = None


# Start with the best model from the previous training
best_model = solver.model
best_val = max(solver.val_acc_history)

# Hyperparameter combinations to try
configs = [
    (100, 1e-3, 0.0),
    (100, 5e-4, 0.1),
    (200, 1e-3, 0.1),
    (200, 5e-4, 0.5),
]

# Train and evaluate different models
for hidden_dim, lr, reg in configs:

    model_tuned = TwoLayerNet(
        input_dim=input_size,
        hidden_dim=hidden_dim,
        num_classes=num_classes,
```

```
        weight_scale=1e-3,
        reg=reg
    )

    solver_tuned = Solver(
        model_tuned,
        data,
        update_rule='sgd',
        optim_config={
            'learning_rate': lr
        },
        lr_decay=0.95,
        num_epochs=10,
        batch_size=100,
        print_every=100,
        verbose=False
    )

    solver_tuned.train()

    val_acc = max(solver_tuned.val_acc_history)

    print(
        "hidden_dim=%d, lr=%e, reg=%f, val_acc=%f"
        % (hidden_dim, lr, reg, val_acc)
    )

    # Keep the model with the highest validation accuracy
    if val_acc > best_val:
        best_val = val_acc
        best_model = solver_tuned.model

print("Best validation accuracy:", best_val)
```

```
hidden_dim=100, lr=1.000000e-03, reg=0.000000, val_acc=0.519000
hidden_dim=100, lr=5.000000e-04, reg=0.100000, val_acc=0.534000
hidden_dim=200, lr=1.000000e-03, reg=0.100000, val_acc=0.528000
hidden_dim=200, lr=5.000000e-04, reg=0.500000, val_acc=0.527000
Best validation accuracy: 0.534
```

12 Test your model!

Run your best model on the validation and test sets. You should achieve above 48% accuracy on the validation set and the test set.

[18]:
```
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
```

```
Validation set accuracy:  0.534
```

[19]:
```
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Test set accuracy:  0.496
```

[20]:
```
# Save best model
best_model.save("best_two_layer_net.npy")
```

```
best_two_layer_net.npy saved.
```

12.1 Inline Question 2:

Now that you have trained a Neural Network classifier, you may find that your testing accuracy is much lower than the training accuracy. In what ways can we decrease this gap? Select all that apply.

1. Train on a larger dataset.
2. Add more hidden units.
3. Increase the regularization strength.
4. None of the above.

Answer: 1 and 3.

Explanation:

1. Training on a larger dataset can reduce overfitting by helping the model generalize better to unseen data.
2. Adding more hidden units increases model capacity and may lead to more overfitting, potentially increasing the gap between training and test accuracy.
3. Increasing regularization strength discourages overly complex models and can reduce overfitting, helping to narrow the gap between training and test accuracy.

features

November 10, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# assignment folder, e.g. 'icv83551/assignments/assignment1/'
FOLDERNAME = 'icv83551/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/icv83551/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
/content/drive/My Drive/icv83551/assignments/assignment1/icv83551/datasets
/content/drive/My Drive/icv83551/assignments/assignment1
```

1 Image features exercise

*Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission.

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image. In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of your work for this exercise will be done in this notebook.

[2]:
```
import random
import numpy as np
```

```
from icv83551.data_utils import load_CIFAR10
import matplotlib.pyplot as plt


%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

#%load_ext autoreload
#%autoreload 2
```

1.1 Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

[3]:
```
from icv83551.features import color_histogram_hsv, hog_feature

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000):
    # Load the raw CIFAR-10 data
    cifar10_dir = 'icv83551/datasets/cifar-10-batches-py'

    # Cleaning up variables to prevent loading data multiple times (which may␣
 ↪cause memory issue)
    try:
       del X_train, y_train
       del X_test, y_test
       print('Clear previously loaded data.')
    except:
       pass

    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # Subsample the data
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]

    return X_train, y_train, X_val, y_val, X_test, y_test

X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()
```

1.2 Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your own interest.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The extract_features function takes a set of images and a list of feature functions and evaluates each feature function on each image, storing the results in a matrix where each column is the concatenation of all feature vectors for a single image.

[4]:
```
from icv83551.features import *

# num_color_bins = 10 # Number of bins in the color histogram
num_color_bins = 25 # Number of bins in the color histogram
feature_fns = [hog_feature, lambda img: color_histogram_hsv(img,␣
 ↪nbin=num_color_bins)]
X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)

# Preprocessing: Subtract the mean feature
mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

# Preprocessing: Divide by standard deviation. This ensures that each feature
# has roughly the same scale.
std_feat = np.std(X_train_feats, axis=0, keepdims=True)
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat

# Preprocessing: Add a bias dimension
X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])
```

```
Done extracting features for 1000 / 49000 images
Done extracting features for 2000 / 49000 images
Done extracting features for 3000 / 49000 images
Done extracting features for 4000 / 49000 images
Done extracting features for 5000 / 49000 images
```

```
Done extracting features for 6000 / 49000 images
Done extracting features for 7000 / 49000 images
Done extracting features for 8000 / 49000 images
Done extracting features for 9000 / 49000 images
Done extracting features for 10000 / 49000 images
Done extracting features for 11000 / 49000 images
Done extracting features for 12000 / 49000 images
Done extracting features for 13000 / 49000 images
Done extracting features for 14000 / 49000 images
Done extracting features for 15000 / 49000 images
Done extracting features for 16000 / 49000 images
Done extracting features for 17000 / 49000 images
Done extracting features for 18000 / 49000 images
Done extracting features for 19000 / 49000 images
Done extracting features for 20000 / 49000 images
Done extracting features for 21000 / 49000 images
Done extracting features for 22000 / 49000 images
Done extracting features for 23000 / 49000 images
Done extracting features for 24000 / 49000 images
Done extracting features for 25000 / 49000 images
Done extracting features for 26000 / 49000 images
Done extracting features for 27000 / 49000 images
Done extracting features for 28000 / 49000 images
Done extracting features for 29000 / 49000 images
Done extracting features for 30000 / 49000 images
Done extracting features for 31000 / 49000 images
Done extracting features for 32000 / 49000 images
Done extracting features for 33000 / 49000 images
Done extracting features for 34000 / 49000 images
Done extracting features for 35000 / 49000 images
Done extracting features for 36000 / 49000 images
Done extracting features for 37000 / 49000 images
Done extracting features for 38000 / 49000 images
Done extracting features for 39000 / 49000 images
Done extracting features for 40000 / 49000 images
Done extracting features for 41000 / 49000 images
Done extracting features for 42000 / 49000 images
Done extracting features for 43000 / 49000 images
Done extracting features for 44000 / 49000 images
Done extracting features for 45000 / 49000 images
Done extracting features for 46000 / 49000 images
Done extracting features for 47000 / 49000 images
Done extracting features for 48000 / 49000 images
Done extracting features for 49000 / 49000 images
```

1.3 Train Softmax classifier on features

Using the Softmax code developed earlier in the assignment, train Softmax classifiers on top of the features extracted above; this should achieve better results than training them directly on top of raw pixels.

[5]:
```
# Use the validation set to tune the learning rate and regularization strength

from icv83551.classifiers.linear_classifier import Softmax

learning_rates = [1e-7, 3e-7, 5e-7]
regularization_strengths = [1e4, 5e4, 1e5]

results = {}
best_val = -1
best_softmax = None


for lr in learning_rates:
    for reg in regularization_strengths:

        # Create and train a Softmax classifier
        softmax = Softmax()

        softmax.train(
            X_train_feats,
            y_train,
            learning_rate=lr,
            reg=reg,
            num_iters=1500,
            verbose=False
        )

        # Predict on training and validation sets
        y_train_pred = softmax.predict(X_train_feats)
        y_val_pred = softmax.predict(X_val_feats)

        # Compute accuracies
        train_accuracy = np.mean(y_train_pred == y_train)
        val_accuracy = np.mean(y_val_pred == y_val)

        # Save results
        results[(lr, reg)] = (train_accuracy, val_accuracy)

        # Keep the best classifier
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = softmax
```

```
# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
                lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved: %f' % best_val)
```

```
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.097286 val accuracy: 0.095000
lr 1.000000e-07 reg 5.000000e+04 train accuracy: 0.420816 val accuracy: 0.430000
lr 1.000000e-07 reg 1.000000e+05 train accuracy: 0.422286 val accuracy: 0.416000
lr 3.000000e-07 reg 1.000000e+04 train accuracy: 0.420327 val accuracy: 0.419000
lr 3.000000e-07 reg 5.000000e+04 train accuracy: 0.422837 val accuracy: 0.423000
lr 3.000000e-07 reg 1.000000e+05 train accuracy: 0.416735 val accuracy: 0.412000
lr 5.000000e-07 reg 1.000000e+04 train accuracy: 0.424490 val accuracy: 0.425000
lr 5.000000e-07 reg 5.000000e+04 train accuracy: 0.408633 val accuracy: 0.397000
lr 5.000000e-07 reg 1.000000e+05 train accuracy: 0.419857 val accuracy: 0.425000
best validation accuracy achieved: 0.430000
```

[6]:
```
# Evaluate your trained Softmax on the test set: you should be able to get at␣
 ↪least 0.42
y_test_pred = best_softmax.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print(test_accuracy)
```

```
0.424
```

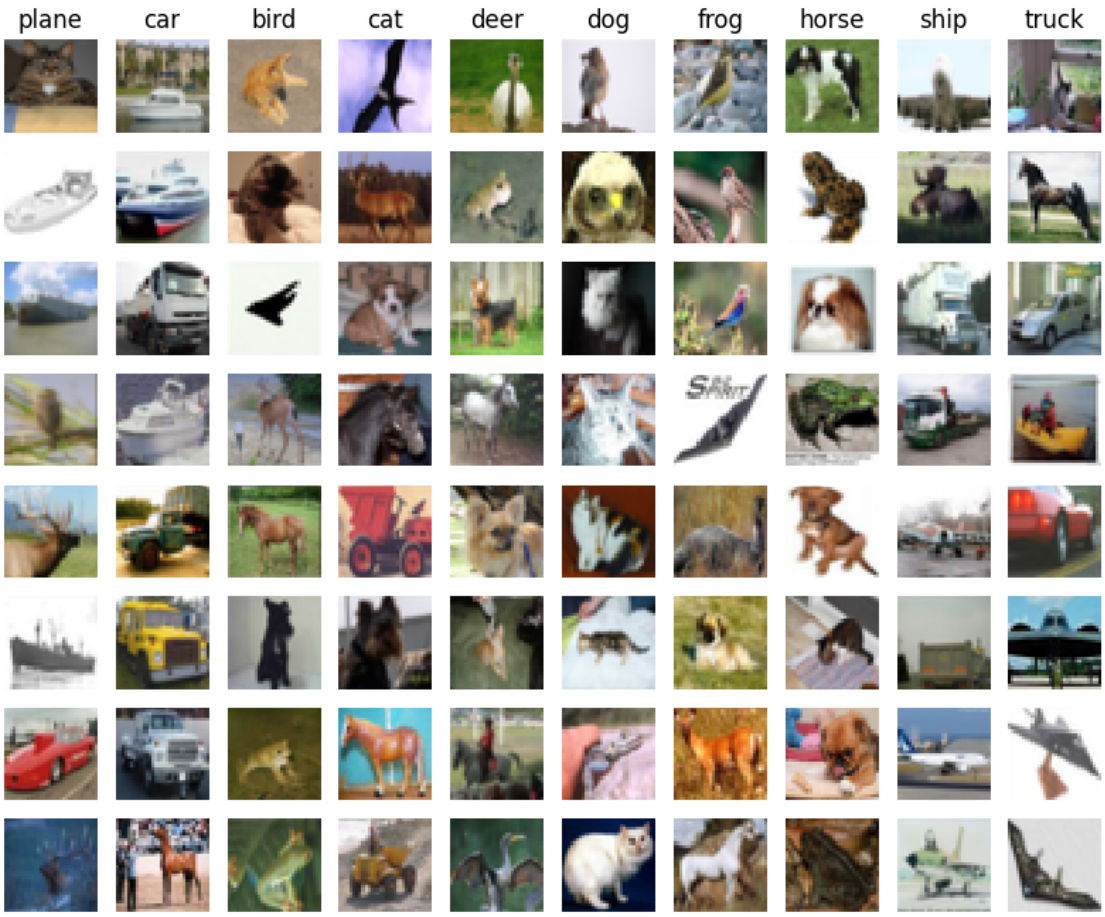
[7]:
```
# Save best softmax model
best_softmax.save("best_softmax_features.npy")
```

```
best_softmax_features.npy saved.
```

[8]:
```
# An important way to gain intuition about how an algorithm works is to
# visualize the mistakes that it makes. In this visualization, we show examples
# of images that are misclassified by our current system. The first column
# shows images that our system labeled as "plane" but whose true label is
# something other than "plane".

examples_per_class = 8
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']
for cls, cls_name in enumerate(classes):
    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]
    idxs = np.random.choice(idxs, examples_per_class, replace=False)
    for i, idx in enumerate(idxs):
```

```
        plt.subplot(examples_per_class, len(classes), i * len(classes) + cls +␣
↪1)
        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls_name)
plt.show()
```



1.3.1 Inline question 1:

Describe the misclassification results that you see. Do they make sense?

Answer :

The misclassification results show that the classifier sometimes confuses images with similar colors, shapes, or textures. Some mistakes are understandable because different object classes can share similar visual features. However, some predictions seem incorrect even when the objects are clearly distinguishable to humans.

These errors make sense because the classifier relies on HOG and color histogram features, which

capture edges, textures, and color distributions but do not fully represent the spatial structure or semantic meaning of objects. Therefore, images with similar low-level features may be assigned to the wrong class.

1.4 Neural Network on image features

Earlier in this assigment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels. In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features. This approach should outperform all previous approaches: you should easily be able to achieve over 55% classification accuracy on the test set; our best model achieves about 60% classification accuracy.

[9]:
```
# Preprocessing: Remove the bias dimension
# Make sure to run this cell only ONCE
print(X_train_feats.shape)
X_train_feats = X_train_feats[:, :-1]
X_val_feats = X_val_feats[:, :-1]
X_test_feats = X_test_feats[:, :-1]

print(X_train_feats.shape)
```

```
(49000, 170)
(49000, 169)
```

[10]:
```
from icv83551.classifiers.fc_net import TwoLayerNet
from icv83551.solver import Solver

input_dim = X_train_feats.shape[1]
hidden_dim = 750
num_classes = 10

data = {
    'X_train': X_train_feats,
    'y_train': y_train,
    'X_val': X_val_feats,
    'y_val': y_val,
    'X_test': X_test_feats,
    'y_test': y_test,
}

best_net = None


# Train a two-layer neural network on image features
np.random.seed(231)

net = TwoLayerNet(
```

```
    input_dim,
    hidden_dim,
    num_classes,
    weight_scale=1e-2,
    reg=1e-3
)

solver = Solver(
    net,
    data,
    update_rule="adam",
    optim_config={"learning_rate": 5e-4},
    lr_decay=0.95,
    num_epochs=25,
    batch_size=200,
    print_every=200,
    verbose=True
)

solver.train()

best_net = solver.model

val_scores = best_net.loss(data["X_val"])
val_predictions = np.argmax(val_scores, axis=1)
val_accuracy = np.mean(val_predictions == data["y_val"])

print("Best validation accuracy:", val_accuracy)
```

```
(Iteration 1 / 6125) loss: 2.306700
(Epoch 0 / 25) train acc: 0.188000; val_acc: 0.207000
(Iteration 201 / 6125) loss: 1.201056
(Epoch 1 / 25) train acc: 0.510000; val_acc: 0.519000
(Iteration 401 / 6125) loss: 1.194569
(Epoch 2 / 25) train acc: 0.613000; val_acc: 0.549000
(Iteration 601 / 6125) loss: 1.240404
(Epoch 3 / 25) train acc: 0.617000; val_acc: 0.561000
(Iteration 801 / 6125) loss: 1.198830
(Epoch 4 / 25) train acc: 0.628000; val_acc: 0.580000
(Iteration 1001 / 6125) loss: 1.170826
(Iteration 1201 / 6125) loss: 1.057959
(Epoch 5 / 25) train acc: 0.641000; val_acc: 0.607000
(Iteration 1401 / 6125) loss: 1.204120
(Epoch 6 / 25) train acc: 0.644000; val_acc: 0.596000
(Iteration 1601 / 6125) loss: 1.149209
(Epoch 7 / 25) train acc: 0.669000; val_acc: 0.608000
(Iteration 1801 / 6125) loss: 1.156779
(Epoch 8 / 25) train acc: 0.680000; val_acc: 0.606000
```

```
(Iteration 2001 / 6125) loss: 1.100426
(Iteration 2201 / 6125) loss: 0.977895
(Epoch 9 / 25) train acc: 0.714000; val_acc: 0.617000
(Iteration 2401 / 6125) loss: 0.962654
(Epoch 10 / 25) train acc: 0.712000; val_acc: 0.633000
(Iteration 2601 / 6125) loss: 0.916324
(Epoch 11 / 25) train acc: 0.722000; val_acc: 0.613000
(Iteration 2801 / 6125) loss: 0.971558
(Epoch 12 / 25) train acc: 0.748000; val_acc: 0.619000
(Iteration 3001 / 6125) loss: 0.988367
(Epoch 13 / 25) train acc: 0.715000; val_acc: 0.607000
(Iteration 3201 / 6125) loss: 0.861120
(Iteration 3401 / 6125) loss: 0.930011
(Epoch 14 / 25) train acc: 0.742000; val_acc: 0.619000
(Iteration 3601 / 6125) loss: 0.951818
(Epoch 15 / 25) train acc: 0.724000; val_acc: 0.616000
(Iteration 3801 / 6125) loss: 0.842042
(Epoch 16 / 25) train acc: 0.750000; val_acc: 0.625000
(Iteration 4001 / 6125) loss: 0.849042
(Epoch 17 / 25) train acc: 0.737000; val_acc: 0.613000
(Iteration 4201 / 6125) loss: 0.815797
(Iteration 4401 / 6125) loss: 0.911091
(Epoch 18 / 25) train acc: 0.786000; val_acc: 0.613000
(Iteration 4601 / 6125) loss: 0.850766
(Epoch 19 / 25) train acc: 0.768000; val_acc: 0.612000
(Iteration 4801 / 6125) loss: 0.861580
(Epoch 20 / 25) train acc: 0.779000; val_acc: 0.620000
(Iteration 5001 / 6125) loss: 0.854597
(Epoch 21 / 25) train acc: 0.785000; val_acc: 0.617000
(Iteration 5201 / 6125) loss: 0.796073
(Epoch 22 / 25) train acc: 0.788000; val_acc: 0.622000
(Iteration 5401 / 6125) loss: 0.796016
(Iteration 5601 / 6125) loss: 0.835056
(Epoch 23 / 25) train acc: 0.803000; val_acc: 0.617000
(Iteration 5801 / 6125) loss: 0.890977
(Epoch 24 / 25) train acc: 0.775000; val_acc: 0.613000
(Iteration 6001 / 6125) loss: 0.882952
(Epoch 25 / 25) train acc: 0.793000; val_acc: 0.615000
Best validation accuracy: 0.633
```

[11]:
```python
# Run your best neural net classifier on the test set. You should be able
# to get more than 58% accuracy. It is also possible to get >60% accuracy
# with careful tuning.

y_test_pred = np.argmax(best_net.loss(data['X_test']), axis=1)
test_acc = (y_test_pred == data['y_test']).mean()
print(test_acc)
```

```
0.592
```

[12]:
```
# Save best model
best_net.save("best_two_layer_net_features.npy")
```

```
best_two_layer_net_features.npy saved.
```

FullyConnectedNets

November 10, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# assignment folder, e.g. 'icv83551/assignments/assignment1/'
FOLDERNAME = 'icv83551/assignments/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/icv83551/datasets/
# %cd /content/drive/My\ Drive/$FOLDERNAME
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/My Drive/icv83551/assignments/assignment1/icv83551/datasets
/content/drive/My Drive/icv83551/assignments/assignment1
```

1 Multi-Layer Fully Connected Network

In this exercise, you will implement a fully connected network with an arbitrary number of hidden layers.

[2]:
```
# from google.colab import drive
# drive.mount('/content/drive')
```

Read through the `FullyConnectedNet` class in the file `icv83551/classifiers/fc_net.py`.

Implement the network initialization, forward pass, and backward pass. Throughout this assignment, you will be implementing layers in `icv83551/layers.py`. You can re-use your implementations for `affine_forward`, `affine_backward`, `relu_forward`, `relu_backward`, and `softmax_loss`

from before. For right now, don't worry about implementing dropout or batch/layer normalization yet, as you will add those features later.

[3]:
```
# Setup cell.
import time
import numpy as np
import matplotlib.pyplot as plt
from icv83551.classifiers.fc_net import *
from icv83551.data_utils import get_CIFAR10_data
from icv83551.gradient_check import eval_numerical_gradient,␣
 ↪eval_numerical_gradient_array
from icv83551.solver import Solver

%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0)  # Set default size of plots.
plt.rcParams["image.interpolation"] = "nearest"
plt.rcParams["image.cmap"] = "gray"

#%load_ext autoreload
#%autoreload 2

def rel_error(x, y):
    """Returns relative error."""
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

[4]:
```
# Load the (preprocessed) CIFAR-10 data.
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print(f"{k}: {v.shape}")
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

1.1 Initial Loss and Gradient Check

As a sanity check, run the following to check the initial loss and to gradient check the network both with and without regularization. This is a good way to see if the initial losses seem reasonable.

For gradient checking, you should expect to see errors around 1e-7 or less.

[5]:
```
np.random.seed(231)
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))
```

```
for reg in [0, 3.14]:
    print("Running check with reg = ", reg)
    model = FullyConnectedNet(
        [H1, H2],
        input_dim=D,
        num_classes=C,
        reg=reg,
        weight_scale=5e-2,
        dtype=np.float64
    )

    loss, grads = model.loss(X, y)
    print("Initial loss: ", loss)

    # Most of the errors should be on the order of e-7 or smaller.
    # NOTE: It is fine however to see an error for W2 on the order of e-5
    # for the check when reg = 0.0
    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name], verbose=False, h=1e-5)
        print(f"{name} relative error: {rel_error(grad_num, grads[name])}")
```

```
Running check with reg =  0
Initial loss:  2.3004790897684924
W1 relative error: 1.4839894098713283e-07
W2 relative error: 2.2120479295080622e-05
W3 relative error: 3.527252852151791e-07
b1 relative error: 5.376386228531692e-09
b2 relative error: 2.085654276112763e-09
b3 relative error: 5.7957243458479405e-11
Running check with reg =  3.14
Initial loss:  7.052114776533016
W1 relative error: 3.904541941902138e-09
W2 relative error: 6.86942277940646e-08
W3 relative error: 2.131129859578198e-08
b1 relative error: 1.4752427965311745e-08
b2 relative error: 1.7223751746766738e-09
b3 relative error: 1.5702714832602802e-10
```

As another sanity check, make sure your network can overfit on a small dataset of 50 images. First, we will try a three-layer network with 100 units in each hidden layer. In the following cell, tweak the **learning rate** and **weight initialization scale** to overfit and achieve 100% training accuracy within 20 epochs.

[6]:
```
# tweaking just the learning rate and initialization scale.

num_train = 50
```

```
small_data = {
  "X_train": data["X_train"][:num_train],
  "y_train": data["y_train"][:num_train],
  "X_val": data["X_val"],
  "y_val": data["y_val"],
}

weight_scale = 1e-2
learning_rate = 5e-3

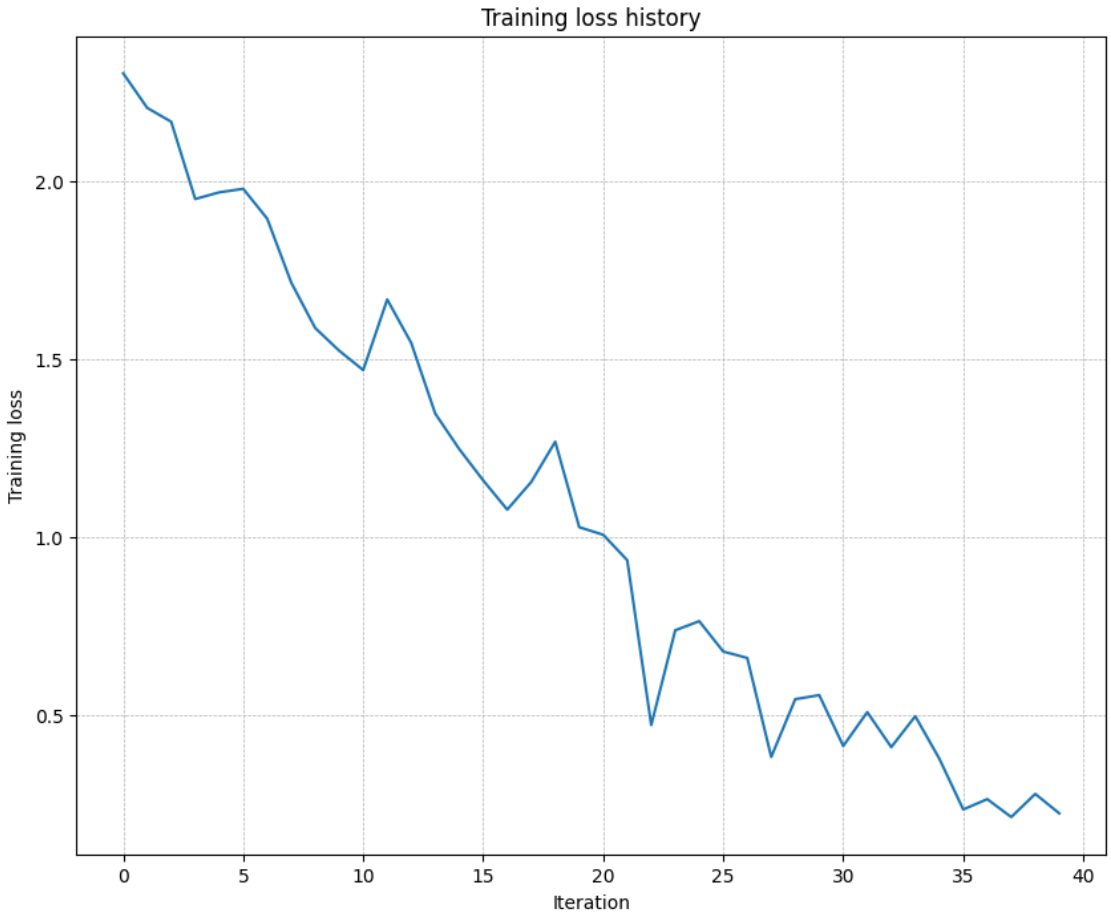
np.random.seed(231)

model = FullyConnectedNet(
    [100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule="sgd",
    optim_config={"learning_rate": learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title("Training loss history")
plt.xlabel("Iteration")
plt.ylabel("Training loss")
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 2.301506
(Epoch 0 / 20) train acc: 0.220000; val_acc: 0.098000
(Epoch 1 / 20) train acc: 0.320000; val_acc: 0.113000
(Epoch 2 / 20) train acc: 0.520000; val_acc: 0.155000
(Epoch 3 / 20) train acc: 0.580000; val_acc: 0.149000
(Epoch 4 / 20) train acc: 0.520000; val_acc: 0.163000
(Epoch 5 / 20) train acc: 0.620000; val_acc: 0.150000
(Iteration 11 / 40) loss: 1.468578
(Epoch 6 / 20) train acc: 0.660000; val_acc: 0.140000
(Epoch 7 / 20) train acc: 0.760000; val_acc: 0.164000
(Epoch 8 / 20) train acc: 0.740000; val_acc: 0.168000
(Epoch 9 / 20) train acc: 0.820000; val_acc: 0.178000
(Epoch 10 / 20) train acc: 0.820000; val_acc: 0.176000
```

```
(Iteration 21 / 40) loss: 1.006394
(Epoch 11 / 20) train acc: 0.880000; val_acc: 0.153000
(Epoch 12 / 20) train acc: 0.840000; val_acc: 0.173000
(Epoch 13 / 20) train acc: 0.960000; val_acc: 0.172000
(Epoch 14 / 20) train acc: 0.920000; val_acc: 0.166000
(Epoch 15 / 20) train acc: 0.920000; val_acc: 0.164000
(Iteration 31 / 40) loss: 0.413586
(Epoch 16 / 20) train acc: 0.980000; val_acc: 0.178000
(Epoch 17 / 20) train acc: 0.940000; val_acc: 0.194000
(Epoch 18 / 20) train acc: 0.960000; val_acc: 0.184000
(Epoch 19 / 20) train acc: 0.980000; val_acc: 0.180000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.180000
```



Now, try to use a five-layer network with 100 units on each layer to overfit on 50 training examples. Again, you will have to adjust the learning rate and weight initialization scale, but you should be able to achieve 100% training accuracy within 20 epochs.

[7]:
```
# tweaking just the learning rate and initialization scale.

num_train = 50
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}

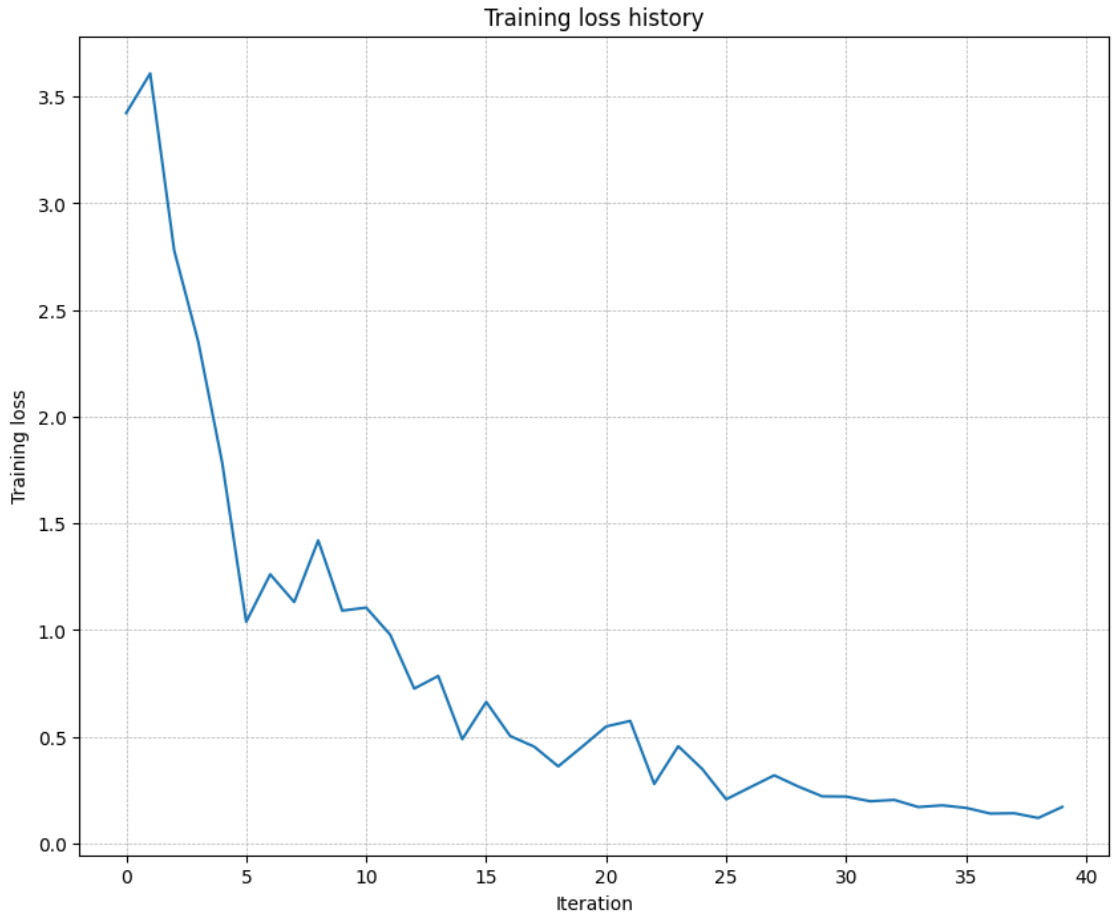

learning_rate = 3e-3
weight_scale = 5e-2


model = FullyConnectedNet(
    [100, 100, 100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule='sgd',
    optim_config={'learning_rate': learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 3.423026
(Epoch 0 / 20) train acc: 0.160000; val_acc: 0.098000
(Epoch 1 / 20) train acc: 0.240000; val_acc: 0.105000
(Epoch 2 / 20) train acc: 0.460000; val_acc: 0.107000
(Epoch 3 / 20) train acc: 0.620000; val_acc: 0.112000
(Epoch 4 / 20) train acc: 0.700000; val_acc: 0.115000
(Epoch 5 / 20) train acc: 0.720000; val_acc: 0.118000
(Iteration 11 / 40) loss: 1.105536
(Epoch 6 / 20) train acc: 0.860000; val_acc: 0.127000
(Epoch 7 / 20) train acc: 0.900000; val_acc: 0.148000
```

```
(Epoch 8 / 20) train acc: 0.880000; val_acc: 0.124000
(Epoch 9 / 20) train acc: 0.960000; val_acc: 0.134000
(Epoch 10 / 20) train acc: 0.960000; val_acc: 0.131000
(Iteration 21 / 40) loss: 0.548986
(Epoch 11 / 20) train acc: 1.000000; val_acc: 0.124000
(Epoch 12 / 20) train acc: 1.000000; val_acc: 0.135000
(Epoch 13 / 20) train acc: 0.980000; val_acc: 0.134000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.144000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.141000
(Iteration 31 / 40) loss: 0.220147
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.137000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.140000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.130000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.145000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.137000
```



1.2 Inline Question 1:

Did you notice anything about the comparative difficulty of training the three-layer network vs. training the five-layer network? In particular, based on your experience, which network seemed more sensitive to the initialization scale? Why do you think that is the case?

1.3 Answer:

[The five-layer network was more sensitive to the weight initialization scale than the three-layer network.

In our experiments, the three-layer network was able to learn with a weight scale of 1e-2, while the five-layer network barely learned with the initial weight scale of 1e-5. We had to increase the weight scale to 5e-2 to successfully reach 100% training accuracy.

This happens because deeper networks involve more layers of computations, and very small initial weights can cause gradients to become extremely small as they propagate backward through the network (vanishing gradients). As a result, the earlier layers learn very slowly.

Therefore, choosing an appropriate initialization scale becomes more important as the network gets deeper.]

2 Update rules

So far we have used vanilla stochastic gradient descent (SGD) as our update rule. More sophisticated update rules can make it easier to train deep networks. We will implement a few of the most commonly used update rules and compare them to vanilla SGD.

2.1 SGD+Momentum

Stochastic gradient descent with momentum is a widely used update rule that tends to make deep networks converge faster than vanilla stochastic gradient descent.

Open the file `icv83551/optim.py` and read the documentation at the top of the file to make sure you understand the API. Implement the SGD+momentum update rule in the function `sgd_momentum` and run the following to check your implementation. You should see errors less than e-8.

[8]:
```
from icv83551.optim import sgd_momentum

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
v = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {"learning_rate": 1e-3, "velocity": v}
next_w, _ = sgd_momentum(w, dw, config=config)

expected_next_w = np.asarray([
  [ 0.1406,      0.20738947,  0.27417895,  0.34096842,  0.40775789],
  [ 0.47454737,  0.54133684,  0.60812632,  0.67491579,  0.74170526],
```

```
  [ 0.80849474,  0.87528421,  0.94207368,  1.00886316,  1.07565263],
  [ 1.14244211,  1.20923158,  1.27602105,  1.34281053,  1.4096    ]])
expected_velocity = np.asarray([
  [ 0.5406,      0.55475789,  0.56891579, 0.58307368,  0.59723158],
  [ 0.61138947,  0.62554737,  0.63970526,  0.65386316,  0.66802105],
  [ 0.68217895,  0.69633684,  0.71049474,  0.72465263,  0.73881053],
  [ 0.75296842,  0.76712632,  0.78128421,  0.79544211,  0.8096    ]])

# Should see relative errors around e-8 or less
print("next_w error: ", rel_error(next_w, expected_next_w))
print("velocity error: ", rel_error(expected_velocity, config["velocity"]))
```

```
next_w error:  8.882347033505819e-09
velocity error:  4.269287743278663e-09
```

Once you have done so, run the following to train a six-layer network with both SGD and SGD+momentum. You should see the SGD+momentum update rule converge faster.

[9]:
```
num_train = 4000
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}

solvers = {}

for update_rule in ['sgd', 'sgd_momentum']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )

    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': 5e-3},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))
```

```
axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

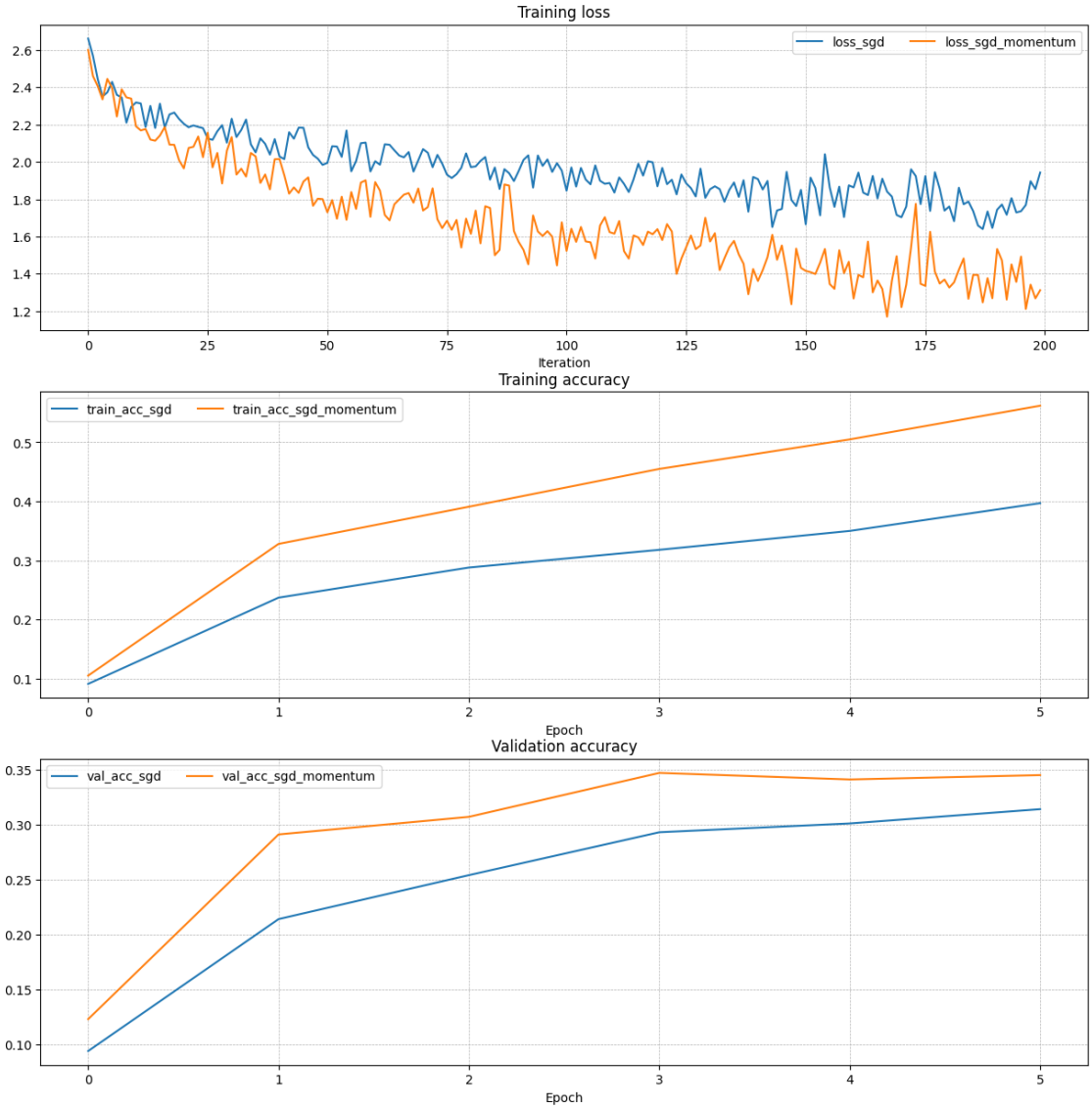
for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"loss_{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"train_acc_{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"val_acc_{update_rule}")

for ax in axes:
    ax.legend(loc="best", ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()
```

```
Running with  sgd
(Iteration 1 / 200) loss: 2.661472
(Epoch 0 / 5) train acc: 0.091000; val_acc: 0.094000
(Iteration 11 / 200) loss: 2.318900
(Iteration 21 / 200) loss: 2.204332
(Iteration 31 / 200) loss: 2.231509
(Epoch 1 / 5) train acc: 0.237000; val_acc: 0.214000
(Iteration 41 / 200) loss: 2.028832
(Iteration 51 / 200) loss: 1.994833
(Iteration 61 / 200) loss: 2.003613
(Iteration 71 / 200) loss: 2.068706
(Epoch 2 / 5) train acc: 0.288000; val_acc: 0.254000
(Iteration 81 / 200) loss: 1.972013
(Iteration 91 / 200) loss: 1.951811
(Iteration 101 / 200) loss: 1.845320
(Iteration 111 / 200) loss: 1.835569
(Epoch 3 / 5) train acc: 0.318000; val_acc: 0.293000
(Iteration 121 / 200) loss: 1.968026
(Iteration 131 / 200) loss: 1.853375
(Iteration 141 / 200) loss: 1.907624
(Iteration 151 / 200) loss: 1.665108
(Epoch 4 / 5) train acc: 0.350000; val_acc: 0.301000
(Iteration 161 / 200) loss: 1.863153
(Iteration 171 / 200) loss: 1.703192
(Iteration 181 / 200) loss: 1.761111
(Iteration 191 / 200) loss: 1.743599
(Epoch 5 / 5) train acc: 0.397000; val_acc: 0.314000
Running with  sgd_momentum
(Iteration 1 / 200) loss: 2.600854
```

```
(Epoch 0 / 5) train acc: 0.105000; val_acc: 0.123000
(Iteration 11 / 200) loss: 2.190892
(Iteration 21 / 200) loss: 1.965249
(Iteration 31 / 200) loss: 2.133140
(Epoch 1 / 5) train acc: 0.328000; val_acc: 0.291000
(Iteration 41 / 200) loss: 2.015306
(Iteration 51 / 200) loss: 1.729825
(Iteration 61 / 200) loss: 1.891746
(Iteration 71 / 200) loss: 1.739294
(Epoch 2 / 5) train acc: 0.391000; val_acc: 0.307000
(Iteration 81 / 200) loss: 1.614299
(Iteration 91 / 200) loss: 1.570246
(Iteration 101 / 200) loss: 1.522644
(Iteration 111 / 200) loss: 1.615110
(Epoch 3 / 5) train acc: 0.455000; val_acc: 0.347000
(Iteration 121 / 200) loss: 1.581017
(Iteration 131 / 200) loss: 1.573045
(Iteration 141 / 200) loss: 1.361248
(Iteration 151 / 200) loss: 1.415646
(Epoch 4 / 5) train acc: 0.505000; val_acc: 0.341000
(Iteration 161 / 200) loss: 1.267478
(Iteration 171 / 200) loss: 1.221236
(Iteration 181 / 200) loss: 1.326100
(Iteration 191 / 200) loss: 1.532960
(Epoch 5 / 5) train acc: 0.562000; val_acc: 0.345000
```



2.2 RMSProp and Adam

RMSProp [1] and Adam [2] are update rules that set per-parameter learning rates by using a running average of the second moments of gradients.

In the file `icv83551/optim.py`, implement the RMSProp update rule in the `rmsprop` function and implement the Adam update rule in the `adam` function, and check your implementations using the tests below.

NOTE: Please implement the *complete* Adam update rule (with the bias correction mechanism), not the first simplified version mentioned in the course notes.

[1] Tijmen Tieleman and Geoffrey Hinton. "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude." COURSERA: Neural Networks for Machine Learning 4 (2012).

[2] Diederik Kingma and Jimmy Ba, "Adam: A Method for Stochastic Optimization", ICLR 2015.

[10]:
```
# Test RMSProp implementation
from icv83551.optim import rmsprop

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
cache = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'cache': cache}
next_w, _ = rmsprop(w, dw, config=config)

expected_next_w = np.asarray([
  [-0.39223849, -0.34037513, -0.28849239, -0.23659121, -0.18467247],
  [-0.132737,   -0.08078555, -0.02881884,  0.02316247,  0.07515774],
  [ 0.12716641,  0.17918792,  0.23122175,  0.28326742,  0.33532447],
  [ 0.38739248,  0.43947102,  0.49155973,  0.54365823,  0.59576619]])
expected_cache = np.asarray([
  [ 0.5976,      0.6126277,   0.6277108,   0.64284931,  0.65804321],
  [ 0.67329252,  0.68859723,  0.70395734,  0.71937285,  0.73484377],
  [ 0.75037008,  0.7659518,   0.78158892,  0.79728144,  0.81302936],
  [ 0.82883269,  0.84469141,  0.86060554,  0.87657507,  0.8926    ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('cache error: ', rel_error(expected_cache, config['cache']))
```

```
next_w error:  9.524687511038133e-08
cache error:  2.6477955807156126e-09
```

[11]:
```
# Test Adam implementation
from icv83551.optim import adam

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
m = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)
v = np.linspace(0.7, 0.5, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'm': m, 'v': v, 't': 5}
next_w, _ = adam(w, dw, config=config)

expected_next_w = np.asarray([
  [-0.40094747, -0.34836187, -0.29577703, -0.24319299, -0.19060977],
  [-0.1380274,  -0.08544591, -0.03286534,  0.01971428,  0.0722929],
  [ 0.1248705,   0.17744702,  0.23002243,  0.28259667,  0.33516969],
  [ 0.38774145,  0.44031188,  0.49288093,  0.54544852,  0.59801459]])
```

```
expected_v = np.asarray([
  [ 0.69966,     0.68908382,  0.67851319,  0.66794809,  0.65738853,],
  [ 0.64683452,  0.63628604,  0.6257431,   0.61520571,  0.60467385,],
  [ 0.59414753,  0.58362676,  0.57311152,  0.56260183,  0.55209767,],
  [ 0.54159906,  0.53110598,  0.52061845,  0.51013645,  0.49966,   ]])
expected_m = np.asarray([
  [ 0.48,        0.49947368,  0.51894737,  0.53842105,  0.55789474],
  [ 0.57736842,  0.59684211,  0.61631579,  0.63578947,  0.65526316],
  [ 0.67473684,  0.69421053,  0.71368421,  0.73315789,  0.75263158],
  [ 0.77210526,  0.79157895,  0.81105263,  0.83052632,  0.85      ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('v error: ', rel_error(expected_v, config['v']))
print('m error: ', rel_error(expected_m, config['m']))
```

```
next_w error:  1.1395691798535431e-07
v error:  4.208314038113071e-09
m error:  4.214963193114416e-09
```

Once you have debugged your RMSProp and Adam implementations, run the following to train a pair of deep networks using these new update rules:

[12]:
```
learning_rates = {'rmsprop': 1e-4, 'adam': 1e-3}
for update_rule in ['adam', 'rmsprop']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )
    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': learning_rates[update_rule]},
        verbose=True
    )
    solvers[update_rule] = solver
    solver.train()
    print()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
```

```
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"{update_rule}")

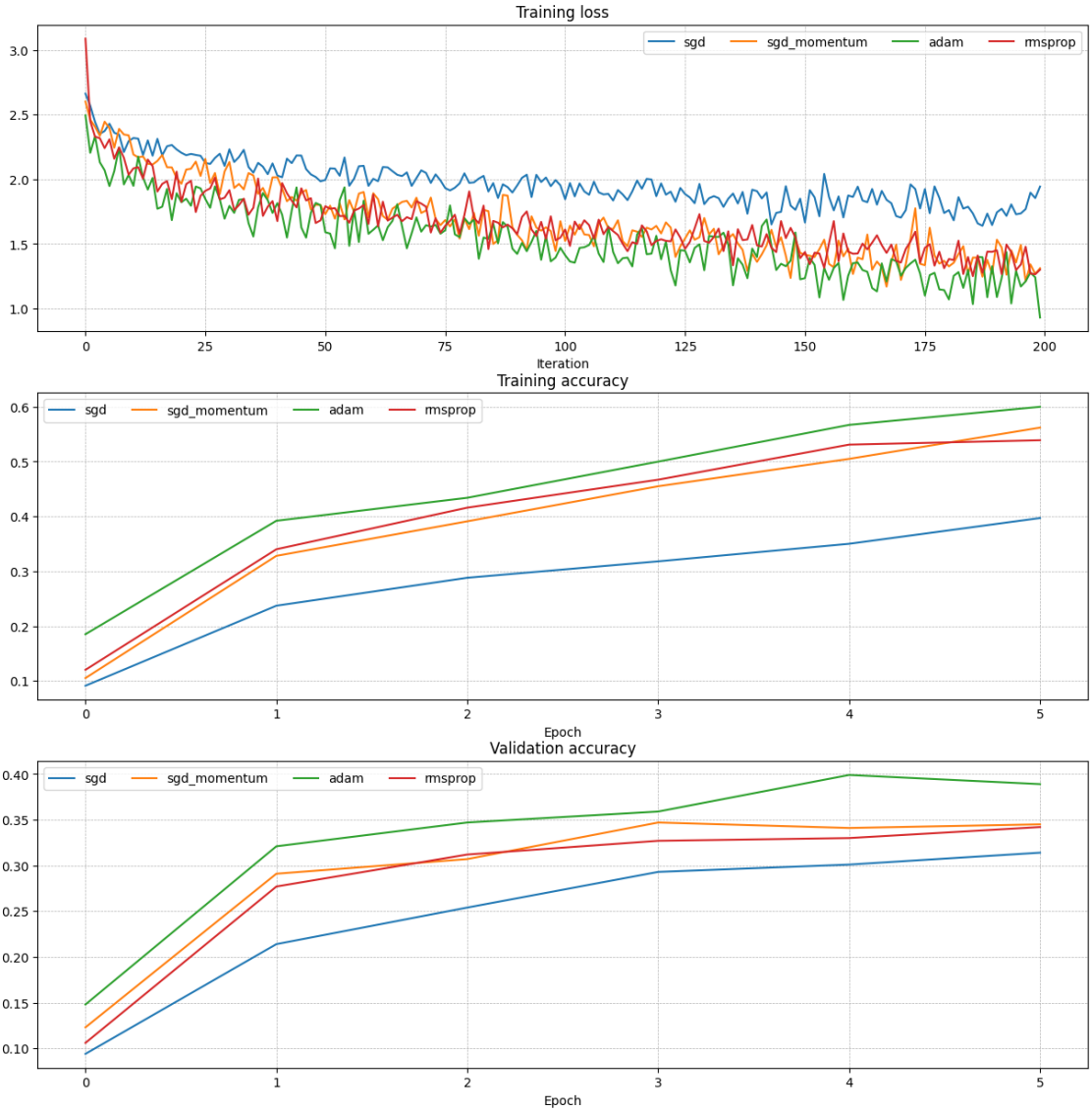
for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()
```

```
Running with  adam
(Iteration 1 / 200) loss: 2.494423
(Epoch 0 / 5) train acc: 0.185000; val_acc: 0.148000
(Iteration 11 / 200) loss: 1.948723
(Iteration 21 / 200) loss: 1.822268
(Iteration 31 / 200) loss: 1.793752
(Epoch 1 / 5) train acc: 0.392000; val_acc: 0.321000
(Iteration 41 / 200) loss: 1.815269
(Iteration 51 / 200) loss: 1.590533
(Iteration 61 / 200) loss: 1.609875
(Iteration 71 / 200) loss: 1.594938
(Epoch 2 / 5) train acc: 0.434000; val_acc: 0.347000
(Iteration 81 / 200) loss: 1.654479
(Iteration 91 / 200) loss: 1.422495
(Iteration 101 / 200) loss: 1.412540
(Iteration 111 / 200) loss: 1.437278
(Epoch 3 / 5) train acc: 0.500000; val_acc: 0.359000
(Iteration 121 / 200) loss: 1.380985
(Iteration 131 / 200) loss: 1.562126
(Iteration 141 / 200) loss: 1.398167
(Iteration 151 / 200) loss: 1.235281
(Epoch 4 / 5) train acc: 0.567000; val_acc: 0.399000
(Iteration 161 / 200) loss: 1.350979
(Iteration 171 / 200) loss: 1.254033
(Iteration 181 / 200) loss: 1.071007
(Iteration 191 / 200) loss: 1.088979
(Epoch 5 / 5) train acc: 0.600000; val_acc: 0.389000

Running with  rmsprop
(Iteration 1 / 200) loss: 3.086694
(Epoch 0 / 5) train acc: 0.120000; val_acc: 0.106000
(Iteration 11 / 200) loss: 2.084156
(Iteration 21 / 200) loss: 1.825993
```

```
(Iteration 31 / 200) loss: 1.915178
(Epoch 1 / 5) train acc: 0.340000; val_acc: 0.277000
(Iteration 41 / 200) loss: 1.676093
(Iteration 51 / 200) loss: 1.791823
(Iteration 61 / 200) loss: 1.882156
(Iteration 71 / 200) loss: 1.812028
(Epoch 2 / 5) train acc: 0.416000; val_acc: 0.312000
(Iteration 81 / 200) loss: 1.906186
(Iteration 91 / 200) loss: 1.574722
(Iteration 101 / 200) loss: 1.611709
(Iteration 111 / 200) loss: 1.599046
(Epoch 3 / 5) train acc: 0.467000; val_acc: 0.327000
(Iteration 121 / 200) loss: 1.535971
(Iteration 131 / 200) loss: 1.509289
(Iteration 141 / 200) loss: 1.488634
(Iteration 151 / 200) loss: 1.437229
(Epoch 4 / 5) train acc: 0.531000; val_acc: 0.330000
(Iteration 161 / 200) loss: 1.423863
(Iteration 171 / 200) loss: 1.355952
(Iteration 181 / 200) loss: 1.383324
(Iteration 191 / 200) loss: 1.451095
(Epoch 5 / 5) train acc: 0.539000; val_acc: 0.342000
```



2.3 Inline Question 2:

AdaGrad, like Adam, is a per-parameter optimization method that uses the following update rule:

```
cache += dw**2
w += - learning_rate * dw / (np.sqrt(cache) + eps)
```

John notices that when he was training a network with AdaGrad that the updates became very small, and that his network was learning slowly. Using your knowledge of the AdaGrad update rule, why do you think the updates would become very small? Would Adam have the same issue?

2.4 Answer:

In AdaGrad, the cache accumulates the squared gradients over all previous iterations. Since squared gradients are non negative, the cache keeps increasing (or stays constant), causing the denominator in the update rule to grow. As a result, the effective learning rate becomes smaller over time, and the weight updates may become extremely small, slowing down training.

Adam does not suffer from this problem in the same way because it uses an exponential moving average of squared gradients instead of accumulating them indefinitely. This allows Adam to adapt to recent gradient values and prevents the learning rate from continuously shrinking due to the accumulation of all past gradients. However, Adam's updates can still become small for other reasons.

3 Train a Good Model!

Train the best fully connected model that you can on CIFAR-10, storing your best model in the `best_model` variable. We require you to get at least 50% accuracy on the validation set using a fully connected network.

If you are careful it should be possible to get accuracies above 55%, but we don't require it for this part and won't assign extra credit for doing so. Later in the next assignment, we will ask you to train the best convolutional network that you can on CIFAR-10, and we would prefer that you spend your effort working on convolutional networks rather than fully connected networks.

Note: In the next assignment, you will learn techniques like BatchNormalization and Dropout which can help you train powerful models.

[15]:
```
best_model = None

# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****

# Train the best fully connected network on CIFAR-10
np.random.seed(231)

model = FullyConnectedNet(
    [512, 512, 256],
    input_dim=3 * 32 * 32,
    num_classes=10,
    weight_scale=1e-2,
    reg=5e-3,
    dtype=np.float32
)

solver = Solver(
    model,
    data,
    update_rule="adam",
    optim_config={"learning_rate": 5e-4},
    lr_decay=0.95,
```

```
    num_epochs=20,
    batch_size=200,
    print_every=200,
    verbose=True
)

solver.train()

# Solver restores the best validation parameters
best_model = solver.model

# Evaluate the best model
val_scores = best_model.loss(data["X_val"])
val_predictions = np.argmax(val_scores, axis=1)
val_accuracy = np.mean(val_predictions == data["y_val"])

print("Best validation accuracy:", val_accuracy)

# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
################################################################################
#                              END OF YOUR CODE                                #
################################################################################
```

```
(Iteration 1 / 4900) loss: 2.806374
(Epoch 0 / 20) train acc: 0.192000; val_acc: 0.197000
(Iteration 201 / 4900) loss: 1.584496
(Epoch 1 / 20) train acc: 0.458000; val_acc: 0.477000
(Iteration 401 / 4900) loss: 1.797793
(Epoch 2 / 20) train acc: 0.497000; val_acc: 0.487000
(Iteration 601 / 4900) loss: 1.569613
(Epoch 3 / 20) train acc: 0.544000; val_acc: 0.505000
(Iteration 801 / 4900) loss: 1.517442
(Epoch 4 / 20) train acc: 0.555000; val_acc: 0.518000
(Iteration 1001 / 4900) loss: 1.482309
(Iteration 1201 / 4900) loss: 1.469138
(Epoch 5 / 20) train acc: 0.574000; val_acc: 0.499000
(Iteration 1401 / 4900) loss: 1.483709
(Epoch 6 / 20) train acc: 0.570000; val_acc: 0.515000
(Iteration 1601 / 4900) loss: 1.296289
(Epoch 7 / 20) train acc: 0.600000; val_acc: 0.516000
(Iteration 1801 / 4900) loss: 1.408090
(Epoch 8 / 20) train acc: 0.653000; val_acc: 0.519000
(Iteration 2001 / 4900) loss: 1.120603
(Iteration 2201 / 4900) loss: 1.273687
(Epoch 9 / 20) train acc: 0.637000; val_acc: 0.533000
(Iteration 2401 / 4900) loss: 1.298839
(Epoch 10 / 20) train acc: 0.663000; val_acc: 0.512000
(Iteration 2601 / 4900) loss: 1.101530
```

```
(Epoch 11 / 20) train acc: 0.688000; val_acc: 0.530000
(Iteration 2801 / 4900) loss: 1.123143
(Epoch 12 / 20) train acc: 0.682000; val_acc: 0.542000
(Iteration 3001 / 4900) loss: 1.141347
(Epoch 13 / 20) train acc: 0.712000; val_acc: 0.550000
(Iteration 3201 / 4900) loss: 1.308587
(Iteration 3401 / 4900) loss: 1.126086
(Epoch 14 / 20) train acc: 0.695000; val_acc: 0.536000
(Iteration 3601 / 4900) loss: 1.044974
(Epoch 15 / 20) train acc: 0.715000; val_acc: 0.568000
(Iteration 3801 / 4900) loss: 0.866332
(Epoch 16 / 20) train acc: 0.750000; val_acc: 0.545000
(Iteration 4001 / 4900) loss: 1.016009
(Epoch 17 / 20) train acc: 0.776000; val_acc: 0.572000
(Iteration 4201 / 4900) loss: 0.925280
(Iteration 4401 / 4900) loss: 1.082693
(Epoch 18 / 20) train acc: 0.821000; val_acc: 0.550000
(Iteration 4601 / 4900) loss: 0.828965
(Epoch 19 / 20) train acc: 0.817000; val_acc: 0.549000
(Iteration 4801 / 4900) loss: 0.967399
(Epoch 20 / 20) train acc: 0.825000; val_acc: 0.554000
Best validation accuracy: 0.572
```

4 Test Your Model!

Run your best model on the validation and test sets. You should achieve at least 50% accuracy on the validation set and the test set.

[16]:
```
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Validation set accuracy:  0.572
Test set accuracy:  0.539
```